



## 5 트리와 그래프

2024-2  
컴퓨터시공학부  
천세진

1

### 목차

7.1 Basic concept of graphs

7.2 여러 가지 그래프

7.3 평면 그래프

7.4 정점의 착색

2

## Preview

### 그래프란?

- ❖ 실생활에서 서로 관련되는 두 가지 또는 그 이상의 양의 관계를 정점과 간선을 이용하여 그림으로 나타낸 것이다.
- ❖ 그래프는 자료 요소들의 관계가 비선형 구조로 나타날 때 사용되는 자료구조이다.

### 그래프의 응용 분야

- ❖ 한붓그리기로 잘 알려진 쾨니히스베르크의 다리 건너기 문제 이후에 그래프 이론은 컴퓨터, 화학, 산업공학, 전자공학, 언어학, 유전학, 사회과학, 경제학 등 많은 분야에 이용되고 있다.
- ❖ 통신 네트워크 전송 문제나 소셜 네트워크(social network)의 분석, 논리 회로의 설계 및 분석, 최단 경로 찾기, 시스템의 흐름도나 컴파일러에서의 최적화에 응용된다.
- ❖ 작업 계획의 분석, 분자 구조식 설계와 화학 결합의 표시, 유한 오토마타, 지하철 노선도, 자동차 내비게이션 시스템, 지도에서 정점들을 색칠하는 문제, 회로도를 평면 회로도로 구현할 수 있는지 등에 이용된다.

### 이 장에서 배우는 내용

- ❖ 그래프의 정의로부터 그래프의 표현 방법, 여러 가지 그래프, 판매원 탐방 문제, 깊이 우선 탐색, 너비우선 탐색, 최단 경로, 평면 그래프, 정점의 착색 등에 대해서 다룬다.

3/99

3

## Chapter 7

## 그래프

# 7.1 Basic concept of Graphs



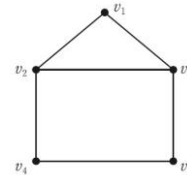
4

## Basic concept of Graphs

### 정의 7-1 그래프

그래프 graph  $G$ 는 순서쌍  $(V, E)$ 로 정의한다. 여기서  $V = \{v_1, v_2, \dots, v_n\}$ 은  $G$ 의 정점 vertex 혹은 노드 node의 집합이고,  $E$ 는 서로 다른 정점의 쌍  $\{v_i, v_j\}$ 의 집합이다. 이러한 쌍들을 간선 edge이라 한다.

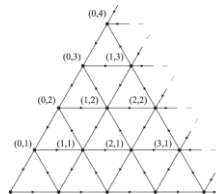
- $G$ 의 정점  $v_1, v_2, \dots, v_n$ 을 나타내기 위해  $n$ 개의 점으로 표시하고,  $\{v_i, v_j\}$ 가  $G$ 의 한 간선이면 정점  $v_i$ 와  $v_j$ 를 선으로 연결한다.
- 정점의 집합  $V = \{v_1, v_2, v_3, v_4, v_5\}$ 와 간선의 집합  $E = \{\{v_1, v_2\}, \{v_1, v_3\}, \{v_2, v_3\}, \{v_2, v_4\}, \{v_3, v_5\}, \{v_4, v_5\}\}$ 로 이루어진 그래프



[그림 7-1] 그래프 예

### 참고

- A graph with an infinite vertex set is called an *infinite graph*. A graph with a finite vertex set is called a *finite graph*. We restrict our attention to finite graphs.



infinite graph  $G(\infty)$

5/99

5

## Multi-graph

### ❖ 다중 그래프 (Multi-graph)

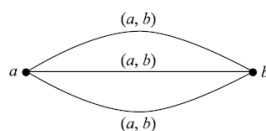
#### 정의 7-2 다중 그래프

그래프  $G = (V, E)$ 에서  $E$ 의 서로 다른 두 개의 정점 사이에 두 개 이상의 간선이 허용되는 그래프를 다중 그래프 multi-graph라 한다.

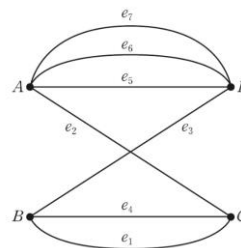
*Multigraphs* may have multiple edges connecting the same two vertices. When  $m$  different edges connect the vertices  $u$  and  $v$ , we say that  $\{u, v\}$  is an edge of *multiplicity*  $m$ .

- 간선  $e_5, e_6, e_7$ 은 같은 정점  $A$ 와  $D$ 에 있는 간선들이다.
- 이는 [정의 7-1]의 그래프 정의에 어긋난다.

- ✓ 그래프  $G$ 의 일부가 [그림 9-1]과 같다면, 간선 집합  $E$ 가  $\{a, b\}, \{a, b\}, \{a, b\}$ 를 부분집합으로 포함



[그림 9-1] 그래프  $G$ 의 일부분



[그림 7-2] 다중 그래프

6/99

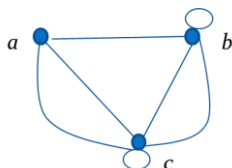
6

# Multi-graph

참고

- An edge that connects a vertex to itself is called a *loop*.
- A *pseudograph* may include loops, as well as multiple edges connecting the same pair of vertices.

- **Example:**  
This pseudograph has both multiple edges and a loop.



**Remark:** There is no standard terminology for graph theory. So, it is crucial that you understand the terminology being used whenever you read material about graphs.

7/99

7

## 그래프의 종류

### 그래프의 종류

- ❖ 무향(무방향) 그래프 (Undirected Graph)
- ❖ 유향(방향) 그래프 (Directed Graph)

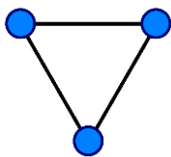
8/99

8

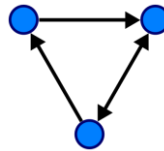
## Undirected graph

### 무향 그래프(undirected graph)

- 그래프  $G$ 에서 정점들의 쌍인 간선  $E$ 를 생각해보자.
- $E_1 = (v_1, v_2)$ ,  $E_2 = (v_2, v_1)$ 라 하면  $E_1$ 과  $E_2$ 는 두 개의 정점  $v_1$ 과  $v_2$ 를 연결하는 간선이다. (여기서  $v_1, v_2$ 는 두 개의 서로 다른 정점이다).
- 두 개의 정점의 순서를 생각하지 않는다면  $E_1$ 과  $E_2$ 는 같은 간선이 되고 두 개의 정점의 순서를 생각한다면  $E_1$ 과  $E_2$ 는 다른 간선이 된다.
- 정점의 순서를 무시하는 그래프  $G = (V, E)$ 를 **무향 그래프** 혹은 **그래프**라 하고, 정점의 순서를 생각하는 그래프를 **유향 그래프**라 한다.



A undirected graph with three vertices and three edges.



A directed graph with three vertices and four directed edges (the double arrow represents an edge in each direction).

9/99

9

## Directed graph

### 유향 그래프(directed graph)

#### 정의 7-3 유향 그래프

유향 그래프 (directed graph; digraph)는 순서쌍  $G = (V, E)$ 이다.  $V$ 는 정점의 집합이고  $E$ 는  $V$ 에 관한 이항 관계이며, 이는 간선의 집합이 된다.

#### ❖ 시점(initial point; source), 종점(terminal point; target)

- 유향 그래프의 정점들의 순서쌍을  $(v_1, v_2)$ 라 할 때  $v_1$ 은 시점이라 하고  $v_2$ 는 종점이라 하며,  $v_1$ 에서  $v_2$ 로의 간선은 화살표( $\rightarrow$ )로 표시한다.

#### 정의 9-5 방향 그래프(Directed Graph: $G = \langle V, E \rangle$ )

화살표로 모서리를 표현해 인접하는 꼭짓점 간의 순서를 알 수 있는 그래프

$$G = \langle V, E \rangle$$

$$V = \{v_1, v_2, \dots, v_n\}$$

$$E = \{e_1, e_2, \dots, e_m\} = \{ \langle v_i, v_j \rangle, \dots \}$$

10/99

10

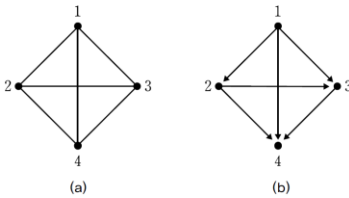
# Directed graph

예제 7-1 무향 그래프와 유향 그래프 그리기

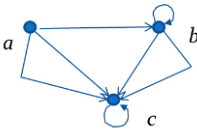
그래프  $G=(V, E)$ 를 무향 그래프와 유향 그래프로 나타내라. 단,  $V=\{1, 2, 3, 4\}$ ,  $E=\{(1, 2), (1, 3), (1, 4), (2, 3), (2, 4), (3, 4)\}$ 이다.

풀이

무향 그래프로 그리면 그림 (a)와 같고, 유향 그래프로 그리면 그림 (b)와 같다.



- A *directed multigraph* may have multiple directed edges. When there are  $m$  directed edges from the vertex  $u$  to the vertex  $v$ , we say that  $(u,v)$  is an edge of *multiplicity*  $m$ .
- **Example:** In this directed multigraph the multiplicity of  $(a,b)$  is 1 and the multiplicity of  $(b,c)$  is 2.



11/99

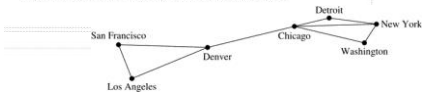
## 참고사항

### Graph Models: Computer Networks

When we build a graph model, we use the appropriate type of graph to capture the important features of the application.

We illustrate this process using graph models of different types of computer networks. In all these graph models, the vertices represent data centers and the edges represent communication links.

To model a computer network where we are only concerned whether two data centers are connected by a communications link, we use a simple graph. This is the appropriate type of graph when we only care whether two data centers are directly linked (and not how many links there may be) and all communications links work in both directions.

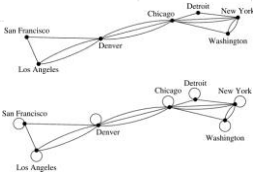


### Graph Models: Computer Networks

To model a computer network where we care about the number of links between data centers, we use a multigraph.

To model a computer network with diagnostic links at data centers, we use a pseudograph, as loops are needed.

To model a network with multiple one-way links, we use a directed multigraph. Note that we could use a directed graph without multiple edges if we only care whether there is at least one link from a data center to another data center.



12/99

Graph Models: Social Networks<sub>1</sub>

Graphs can be used to model social structures based on different kinds of relationships between people or groups.

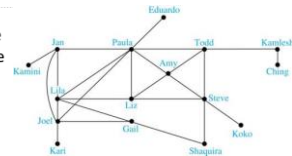
In a *social network*, vertices represent individuals or organizations and edges represent relationships between them.

Useful graph models of social networks include:

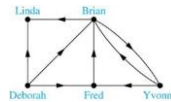
- *friendship graphs* - undirected graphs where two people are connected if they are friends (in the real world, on Facebook, or in a particular virtual world, and so on.)
- *collaboration graphs* - undirected graphs where two people are connected if they collaborate in a specific way
- *influence graphs* - directed graphs where there is an edge from one person to another if the first person can influence the second person

Graph Models: Social Networks<sub>2</sub>

**Example:** A friendship graph where two people are connected if they are Facebook friends.



**Example:** An influence graph



## Examples of Collaboration Graphs

The *Hollywood graph* models the collaboration of actors in films.

- We represent actors by vertices and we connect two vertices if the actors they represent have appeared in the same movie.
- We will study the Hollywood Graph in Section 10.4 when we discuss Kevin Bacon numbers.

An *academic collaboration graph* models the collaboration of researchers who have jointly written a paper in a particular subject.

- We represent researchers in a particular academic discipline using vertices.
- We connect the vertices representing two researchers in this discipline if they are coauthors of a paper.
- We will study the academic collaboration graph for mathematicians when we discuss *Erdős numbers* in Section 10.4.

## Applications to Information Networks

Graphs can be used to model different types of networks that link different types of information.

In a *web graph*, web pages are represented by vertices and links are represented by directed edges.

- A web graph models the web at a particular time.
- We will explain how the web graph is used by search engines in Section 11.4.

In a *citation network*:

- Research papers in a particular discipline are represented by vertices.
- When a paper cites a second paper as a reference, there is an edge from the vertex representing this paper to the vertex representing the second paper.

## Software Design Applications<sub>1</sub>

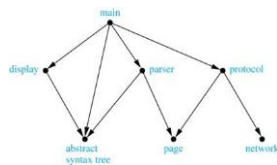
Graph models are extensively used in software design. We will introduce two such models here; one representing the dependency between the modules of a software application and the other representing restrictions in the execution of statements in computer programs.

When a top-down approach is used to design software, the system is divided into modules, each performing a specific task.

We use a *module dependency graph* to represent the dependency between these modules. These dependencies need to be understood before coding can be done.

- In a module dependency graph vertices represent software modules and there is an edge from one module to another if the second module depends on the first.

**Example:** The dependencies between the seven modules in the design of a web browser are represented by this module dependency graph.

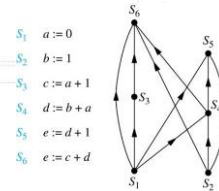


## Software Design Applications<sub>2</sub>

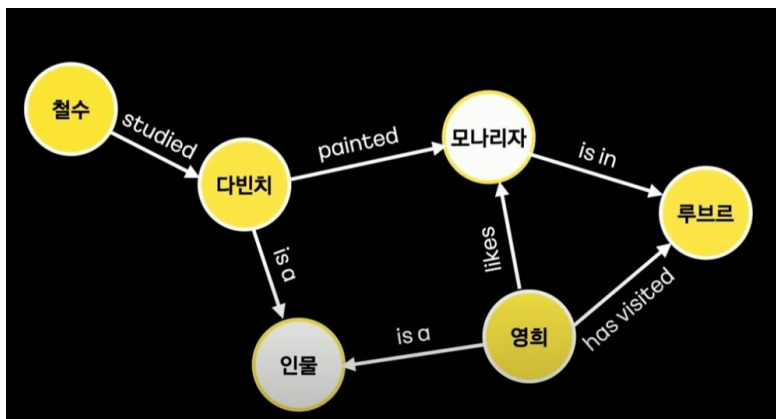
We can use a directed graph called a *precedence graph* to represent which statements must have already been executed before we execute each statement.

- Vertices represent statements in a computer program
- There is a directed edge from a vertex to a second vertex if the second vertex cannot be executed before the first

**Example:** This precedence graph shows which statements must already have been executed before we can execute each of the six statements in the program.



## Knowledge graph



카카오엔터프라이즈가 만드는 지식그래프플랫폼



## Data Graphs

- 지식그래프의 기초
- 개체와 관계를 표현하는 직관적인 추상화
- 데이터 추가를 위한 스키마가 필요없음
- 다양한 소스로부터 데이터를 조립가능한 유연함

17/99

17

## Data Graphs

- 지식그래프의 기초
- 개체와 관계를 표현하는 직관적인 추상화
- 데이터 추가를 위한 스키마가 필요없음
- 다양한 소스로부터 데이터를 조립가능한 유연함
- 추상적인 길이의 패스위에서 질의가 가능
- 그래프 분석 테크닉을 사용 가능함

18/99

18

## 그래프 구조기반 데이터 모델의 유형

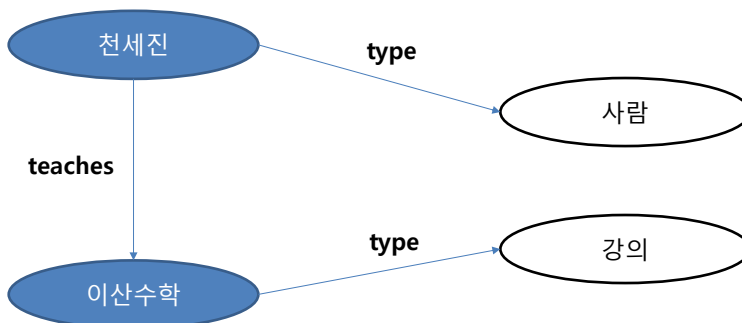
- Directed edge-labelled (multi)graphs (multi-relational graphs)
- Heterogeneous graphs (heterogeneous information network)
- Property graphs
- Hypergraphs (edges connecting set of nodes)
- Hypernodes (nested graphs in a node)

19/99

19

## 그래프 구조기반 데이터 모델의 유형

- Directed edge-labelled (multi)graphs (multi-relational graphs)

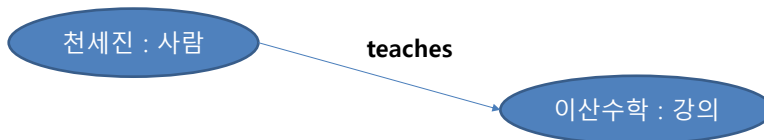


20/99

20

## 그래프 구조기반 데이터 모델의 유형

– Heterogeneous graphs (heterogeneous information network)

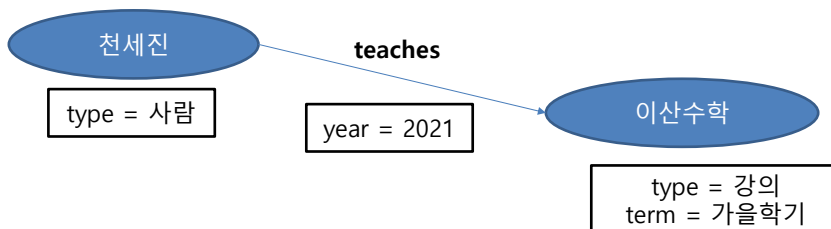


21/99

21

## 그래프 구조기반 데이터 모델의 유형

– Property graphs

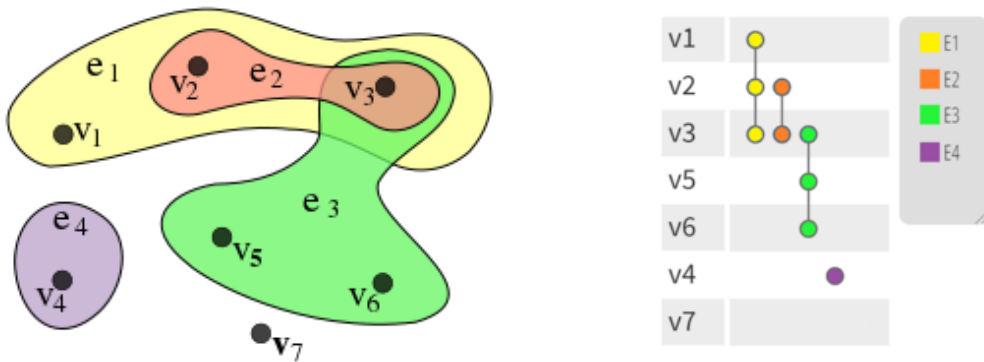


22/99

22

## 그래프 구조기반 데이터 모델의 유형

- Hypergraphs (edges connecting set of nodes)
- Hypernodes (nested graphs in a node)



23/99

23

## Representing Graphs

### 그래프를 표현하는 방법

- ❖ 인접 행렬 (adjacency matrix)
- ❖ 인접 리스트 (adjacency list)

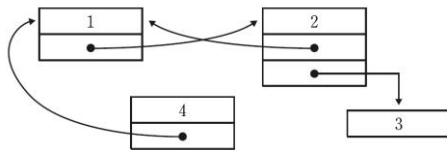
24/99

24

# Representing Graphs : Linked List

## 연결 리스트(linked list)로 표현한 그래프

- 연결 리스트는 **그래프의 각 정점(or Node)**을 그 정점을 표시하는 **레이블(label)**과 다른 정점들을 가리키는 **포인터(pointer)**들의 리스트로 구성되어 있다.
- 각 포인터는 그래프에서의 간선을 나타낸다.
- [그림 7-4]는 [그림 7-3] (c)의 유한 그래프를 연결 리스트로 나타낸 것이다.



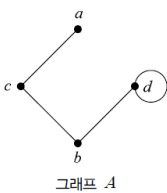
[그림 7-4] 연결 리스트로 표현한 그래프

- 인접 리스트는  $|V|$ 개의 연결 리스트에 실제 간선의 수만큼 원소가 들어있으므로 전체  $O(|V| + |E|)$ 의 기억 공간을 사용한다는 장점이 있고, 어떤 정점을 찾을 때 처음부터 검색해야 하는 단점을 가지고 있다.

25/99

25

# Representing Graphs : Linked List

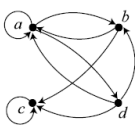
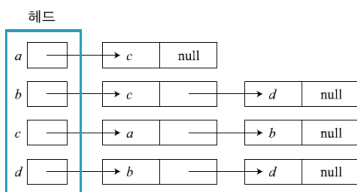


그래프 A

$$G = (V, E)$$

$$V = \{a, b, c, d\}$$

$$E = \{(a, c), (b, c), (b, d), (c, d)\}$$

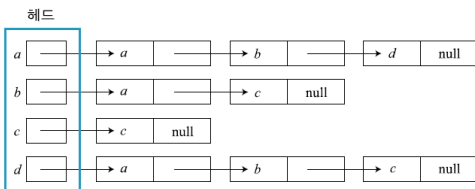


그래프 B

$$G = \langle V, E \rangle$$

$$V = \{a, b, c, d\}$$

$$E = \{ \langle a, a \rangle, \langle a, b \rangle, \langle a, c \rangle, \langle a, d \rangle, \langle b, a \rangle, \langle b, b \rangle, \langle b, c \rangle, \langle b, d \rangle, \langle c, a \rangle, \langle c, b \rangle, \langle c, c \rangle, \langle c, d \rangle, \langle d, a \rangle, \langle d, b \rangle, \langle d, c \rangle, \langle d, d \rangle \}$$



26/99

26

## Representing Graphs : Linked List

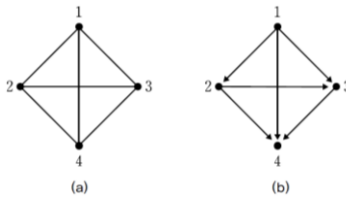
### 예제 7-2 인접 행렬

[예제 7-1]의 두 그래프를 인접 행렬로 표현하라.

#### 풀이

무향 그래프와 유향 그래프를 인접 행렬로 표현하면 다음과 같다.

$$\begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}, \begin{pmatrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$



27/99

27

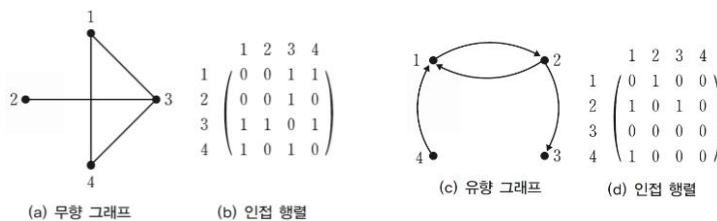
## Representing Graphs : Adjacency Matrix

### 인접 행렬(adjacency matrix)로 표현한 그래프

- 인접 행렬이란 그래프의 연결 관계를 2차원 배열로 나타내는 방법이다.
- 유향 그래프를 인접 행렬로 나타내고자 할 때  $i$ 번째 정점이 시점이고  $j$ 번째 정점이 종점인 간선이 있으면 인접 행렬  $M$ 의  $i$ 행  $j$ 열인  $M[i][j]$ 를 1로 하고, 없으면  $M[i][j]$ 를 0으로 하면 된다.
- 무향 그래프를 인접 행렬로 나타내고자 할 때  $i$ 번째 정점과  $j$ 번째 정점을 연결하는 간선이 존재하면  $M[i][j]$ 와  $M[j][i]$ 를 1로 하고 존재하지 않으면 0으로 하면 된다.
- [그림 7-3]은 이러한 예를 보인 것이다. 그리고  $M[i][j]$ 가 1이라면 정점  $v_i$ 와  $v_j$ 는 인접 정점(adjacent vertex)이라 한다.

If the graphs adjacency matrix is  $A_G = [a_{ij}]$ , then

$$a_{ij} = \begin{cases} 1 & \text{if } \{v_i, v_j\} \text{ is an edge of } G, \\ 0 & \text{otherwise.} \end{cases}$$



28/99

28

## Representing Graphs : Adjacency Matrix

### 인접 행렬(adjacency matrix)로 표현한 그래프

- 만약 가중치가 있는 그래프라면 인접 행렬의 값에 1 대신에 가중치의 값을 직접 넣어주면 된다.
- 무향 그래프에 대한 인접 행렬은 대칭 행렬이다.
- 노드  $i$ 에서 노드  $j$ 로 가는 간선이 있다는 말은 노드  $j$ 에서 노드  $i$ 로 가는 간선도 존재한다는 의미가 된다.
- 따라서, 인접 행렬이 대각 성분  $[adj[i][i]]$ 에서  $i$ 와  $j$ 가 같은 원소들을 기준으로 대칭인 성질을 갖게 된다. [그림 7-3]의 (a) 무향 그래프에 대한 인접 행렬은 [그림 7-3]의 (b)이며 대칭 행렬이다.
- 인접 행렬의 장점은 간선의 수가 많을수록 매우 적합하고, 정점의 번호가 주어지면 배열의 접근만으로 바로 찾을 수 있다는 장점이 있다. 반면에, 간선의 수가 작다면 적합하지 않고 항상  $O(|V|^2)$  만큼의 기억 공간을 사용한다는 단점이 있다.

29/99

29

## Representing Graphs : Adjacency Matrix

Example:



$$\begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

The ordering of vertices is a, b, c, d.



$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

The ordering of vertices is a, b, c, d.

When a graph is sparse, that is, it has few edges relatively to the total number of possible edges, it is much more efficient to represent the graph using an adjacency list than an adjacency matrix. But for a dense graph, which includes a high percentage of possible edges, an adjacency matrix is preferable.

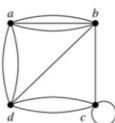
- Note:** The adjacency matrix of a simple graph is symmetric, i.e.,  $a_{ij} = a_{ji}$ .
- Also, since there are no loops, each diagonal entry  $a_{ii}$  for  $i = 1, 2, 3, \dots, n$ , is 0.

Adjacency matrices can also be used to represent graphs with loops and multiple edges.

A loop at the vertex  $v_i$  is represented by a 1 at the  $(i, i)$ th position of the matrix.

When multiple edges connect the same pair of vertices  $v_i$  and  $v_j$ , (or if multiple loops are present at the same vertex), the  $(i, j)$ th entry equals the number of edges connecting the pair of vertices.

**Example:** We give the adjacency matrix of the pseudograph shown here using the ordering of vertices a, b, c, d.



$$\begin{bmatrix} 0 & 3 & 0 & 2 \\ 3 & 0 & 1 & 1 \\ 0 & 1 & 1 & 2 \\ 2 & 1 & 2 & 0 \end{bmatrix}$$

30/99

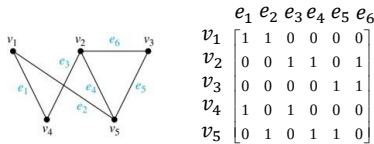
30

## Representing Graphs : Incidence Matrix

- Definition:** Let  $G = (V, E)$  be an undirected graph with vertices where  $v_1, v_2, \dots, v_n$  and edges  $e_1, e_2, \dots, e_m$ . The **incidence matrix** with respect to the ordering of  $V$  and  $E$  is the  $n \times m$  matrix  $M = [m_{ij}]$ , where

$$m_{ij} = \begin{cases} 1 & \text{when edge } e_j \text{ is incident with } v_i, \\ 0 & \text{otherwise.} \end{cases}$$

- Example:** Simple Graph and Incidence Matrix



The rows going from top to bottom represent  $v_1$  through  $v_5$  and the columns going from left to right represent  $e_1$  through  $e_6$ .

- Example:** Pseudograph and Incidence Matrix



The rows going from top to bottom represent  $v_1$  through  $v_5$  and the columns going from left to right represent  $e_1$  through  $e_8$ .

31

31/99

## Degrees of Vertices

### 정점의 차수 및 경로

- 그래프에서 **정점의 차수**는 유한 그래프의 차수와 무한 그래프의 차수로 나눌 수 있다.
- 유한 그래프의 정점의 차수**는 **외차수(out-degree)**와 **내차수(in-degree)**로 나눌 수 있다.
  - ✓ 정점  $a$ 가 시점인 간선의 개수를 정점  $a$ 의 외차수라 하고,
  - ✓ 정점  $a$ 가 종점인 간선의 개수를 정점  $a$ 의 내차수라 한다.
- 무한 그래프의 정점의 차수**는 정점이 가지고 있는 간선의 개수를 말한다.
- 만약 무한 그래프에서 자신의 정점에서 시작해서 다시 자기 자신으로 돌아오는 순환을 가지는 경우 그 정점의 차수는 2이다.

#### 정의 9-7 차수(Degree: $d(v)$ )

꼭짓점  $v$ 에 근접하는 모서리의 수

- 홀수점(Odd Degree): 차수가 홀수인 꼭짓점
- 짝수점(Even Degree): 차수가 짝수인 꼭짓점

방향 그래프에서는

- 외차수(Out-degree:  $out - d(v)$ ): 꼭짓점  $v$ 를 시작으로 하는 화살표의 수
- 내차수(In-degree:  $in - d(v)$ ): 꼭짓점  $v$ 를 끝으로 하는 화살표의 수

32

32/99



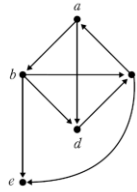
## Degrees of Vertices

### 예제 7-3 유향 그래프의 정점의 차수

오른쪽 유향 그래프의 각 정점에 대한 외차수와 내차수를 구하라.

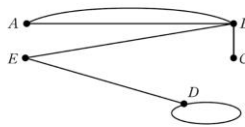
#### 풀이

- 정점  $a$ 의 내차수는 1이고 외차수는 2이다.
- 정점  $b$ 의 내차수는 1이고 외차수는 3이다.
- 정점  $c$ 의 내차수는 2이고 외차수는 2이다.
- 정점  $d$ 의 내차수는 2이고 외차수는 1이다.
- 정점  $e$ 의 내차수는 2이고 외차수는 0이다.



### 예제 7-4 무향 그래프의 정점의 차수

다음의 무향 그래프에서 각 정점의 차수를 구하라.



#### 풀이

- 정점  $A$ 의 차수는 2, 정점  $B$ 의 차수는 4이다.
- 정점  $C$ 의 차수는 1, 정점  $D$ 의 차수는 3이다.
- 정점  $E$ 의 차수는 2이다.

- 무향 그래프에서 모든 정점의 차수의 합은 간선 개수의 두 배가 된다.
- 또한, 차수 0인 정점을 **고립(isolated) 정점**이라고 한다

33/99

33

## Degrees of Vertices (Undirected Graph)

### 정리 9-1 차수에 대한 정리 Undirected graph

- (1) 그래프  $G = (V, E)$ 에서 모든 꼭짓점의 차수의 합은 모서리 수의 두 배이다.

$$\sum_{v \in V} d(v) = 2|E|$$

- (2) 그래프  $G = (V, E)$ 에서 차수가 홀수인 꼭짓점의 수는 짝수이다.

(Handshaking Theorem) : Think about the graph where vertices represent the people at a party and an edge connects two people who have shaken hands.

#### 증명

- (1) 그래프에서 모서리  $e$ 는 항상 두 개의 꼭짓점  $u, v$ 를 연결한다. 그러므로 모서리  $e$ 는 꼭짓점  $u$ 의 차수를 구할 때와 꼭짓점  $v$ 의 차수를 구할 때 두 번 계산된다.  
 $\therefore$  하나의 모서리는 두 개의 꼭짓점의 차수를 계산하는 데에 항상 중복되므로 모든 꼭짓점의 차수의 합은 모서리의 수의 두 배이다.

- (2) 그래프  $G = (V, E)$ 에서 짝수점의 집합이  $V_e$ , 홀수점의 집합이  $V_o$ 일 때, (1)에 의해 다음과 같이 정의된다.

$$2|E| = \sum_{v \in V} d(v) = \sum_{v \in V_e} d(v) + \sum_{v \in V_o} d(v)$$

이때 각  $v \in V_e$ 에 대해  $d(v)$ 가 짝수이므로  $\sum_{v \in V_e} d(v)$ 는 짝수이다.  $\sum_{v \in V_e} d(v)$ 와  $2|E|$ 가 짝수이므로  $\sum_{v \in V_o} d(v)$ 도 짝수가 된다. 그런데 각  $v \in V_o$ 에 대해  $d(v)$ 가 홀수이므로,  $\sum_{v \in V_o} d(v)$ 가 짝수가 되려면  $V_o$ 의 꼭짓점의 개수가 짝수여야 한다.

$\therefore$  차수가 홀수인 꼭짓점의 수는 짝수이다.

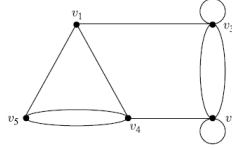
34/99

34

## Degrees of Vertices

### 예제 9-6

다음 그래프의 차수를 이용해 차수에 대한 정리를 확인하라.



풀이

$$G = (V, E)$$

$$V = \{v_1, v_2, v_3, v_4, v_5\}$$

$$E = \{(v_1, v_3), (v_1, v_4), (v_1, v_5), (v_2, v_3), (v_2, v_4), (v_3, v_4), (v_3, v_5), (v_4, v_5), (v_2, v_2), (v_3, v_3), (v_4, v_4), (v_5, v_5)\}$$

$$|E| = 10$$

$$d(v_1) = 3, d(v_2) = 5, d(v_3) = 5, d(v_4) = 4, d(v_5) = 3$$

$$\begin{aligned} \sum_{v \in V} d(v) &= d(v_1) + d(v_2) + d(v_3) + d(v_4) + d(v_5) \\ &= 3 + 5 + 5 + 4 + 3 = 20 = 2 \times 10 = 2|E| \end{aligned}$$

그래프의 각 꼭짓점의 차수의 합은 변의 수의 두 배이다.

차수가 홀수인 점은  $v_1, v_2, v_3, v_5$  네 개로 짝수 개이다.

$\therefore$  그래프의 차수에 대한 정리가 모두 성립한다.

35/99

35

## Degrees of Vertices (Directed Graph)

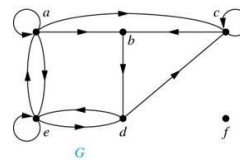
### Directed graph

**Definition:** The *in-degree* of a vertex  $v$ , denoted  $\deg^-(v)$ , is the number of edges which terminate at  $v$ . The *out-degree* of  $v$ , denoted  $\deg^+(v)$ , is the number of edges with  $v$  as their initial vertex. Note that a loop at a vertex contributes 1 to both the in-degree and the out-degree of the vertex.

**Theorem :** Let  $G = (V, E)$  be a graph with directed edges. Then:

$$|E| = \sum_{v \in V} \deg^-(v) = \sum_{v \in V} \deg^+(v).$$

**Proof:** The first sum counts the number of outgoing edges over all vertices and the second sum counts the number of incoming edges over all vertices. It follows that both sums equal the number of edges in the graph.



$$\begin{aligned} \deg^-(a) &= 2, \deg^-(b) = 2, \deg^-(c) = 3, \deg^-(d) = 2, \\ \deg^-(e) &= 3, \deg^-(f) = 0. \end{aligned}$$

$$\begin{aligned} \deg^+(a) &= 4, \deg^+(b) = 1, \deg^+(c) = 2, \deg^+(d) = 2, \\ \deg^+(e) &= 3, \deg^+(f) = 0. \end{aligned}$$

36/99

36

## Path

### 정의 7-4 경로

그래프  $G$ 의 정점  $v_p$ 로부터  $v_q$ 까지의 **경로** path란  $(v_p, v_{i1}), (v_{i1}, v_{i2}), \dots, (v_{in}, v_q)$ 가  $G$ 의 간선일 때를 말하고,  $(v_p, v_{i1}), (v_{i1}, v_{i2}), \dots, (v_{in}, v_q)$ 로 표시하거나  $v_p, v_{i1}, v_{i2}, \dots, v_q$ 로 표시한다.

### 정의 7-5 경로의 길이

**경로의 길이** length란 경로에 있는 간선의 개수를 말한다. 일반적으로 경로를 말할 때는 길이가 얼마인 경로라고 말한다.

- $v_p$ 로부터  $v_q$ 까지의 경로  $v_p, v_{i1}, v_{i2}, \dots, v_q$ 에서  
만일  $v_p \neq v_q$  이면 경로  $v_p, v_{i1}, \dots, v_q$ 를 **개경로(open path)**라 하고  
 $v_p = v_q$  이면 **루프(loop)**라 한다.
- 개경로  $v_p, v_{i1}, v_{i2}, \dots, v_q$ 는  $v_p, v_{i1}, v_{i2}, \dots, v_q$ 가 서로 다른 정점들이면  
**고유 경로(proper path)**라 하고,
- 루프  $v_p, v_{i1}, v_{i2}, \dots, v_q$ 는  $v_p, \dots, v_{in}$ 이 서로 다른 정점들이면 이를 **순환(cycle)**라 한다.

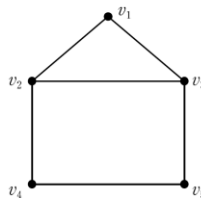
37/99

37

## Path

### 예제 7-5 경로

다음의 그림을 보고 개경로, 루프, 고유 경로, 순환을 찾아라.



### 풀이

$v_1v_3v_5v_4v_2$ 는 길이가 4인 개경로이고,  $v_1v_3v_5v_4v_2v_1$ 은 길이가 6인 루프이다. 그리고  $v_1v_3v_2$ 는 길이가 3인 고유 경로이다. 또한,  $v_1v_2v_3v_1$ 은 길이가 3인 순환이다.

38/99

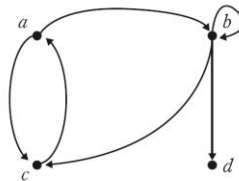
38

# Simple path

### 정의 7-6 단순 경로

단순 경로 simple path는 경로의 시점과 종점은 같을 수 있으나 그 외의 경로상의 모든 정점이 서로 다른 경우의 경로이다. 즉, 고유 경로와 순환은 단순 경로이다.

### ❖ 유형 그래프 예



[그림 7-5] 유형 그래프 예

- $\langle a, c \rangle, \langle a, b, c \rangle, \langle a, c, a \rangle, \langle a, b, b, c \rangle$ 는 각각  $a$ 에서  $c$ 로 가는 경로
- 처음 두 경로는 단순 경로이지만 마지막 두 경로는 단순 경로가 아니다.
- $\langle a, c, a \rangle$ 와  $\langle a, b, c, a \rangle$ 는 순환이다.
- $\langle a, c, a, c, a \rangle$ 와  $\langle a, b, b, c, a \rangle$ 는 루프이지만 순환은 아니다.

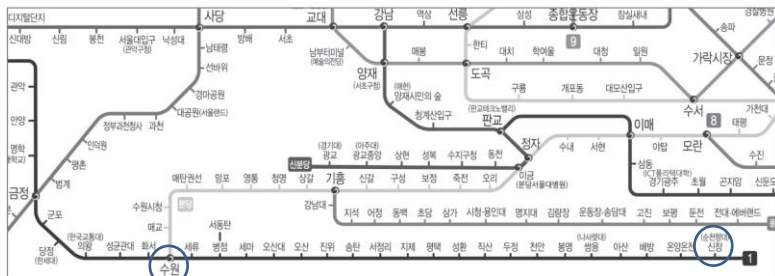
39/99

39

# Simple path

### 예제 7-6 단순 경로, 순환, 루프

버스나 지하철의 노선 안내도는 일종의 무향 그래프이다. 다음의 부분 지하철 노선도에서 수원에서 신창까지의 경로가 단순 경로인지 아닌지, 순환 혹은 루프인지를 보여라.



### 풀이

수원에서 신창까지는 경로상의 모든 정점이 다르므로 단순 경로이다. 순환이나 루프는 시점과 종점이 같아야 하므로 순환이나 루프는 아니다.

40/99

40

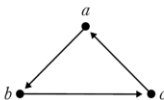
# Connect

## 정의 7-7 연결

정점  $a, b$ 의 쌍이 무향 그래프  $G$ 에서 연결 connected 되어 있다는 것은  $(a, b) \in E$ 이거나  $(b, a) \in E$ 이거나  $(a, c) \in E$ 이고  $(c, b) \in E$ 인 정점  $c$ 가 존재하는 것이다. 즉, 두 정점  $a, b$ 가 연결되어 있다는 것은 정점  $a$ 에서 정점  $b$ 를 연결하는 경로가 있을 때를 말한다. 또한 유향 그래프  $G = (V, E)$ 에서 모든 두 정점  $a, b \in V$  사이에 경로( $a$ 에서  $b$ 로의 경로 및  $b$ 에서  $a$ 로의 경로)가 존재하면  $G$ 는 강하게 연결되었다 strongly connected 라고 한다.

## 예제 7-7 강하게 연결

다음 그래프가 강하게 연결되었는지를 보여라.



## 풀이

유향 그래프에서 임의의 두 정점 사이에 경로가 존재하면 강하게 연결되었다고 한다.  $(a, b)$ ,  $(b, c)$ ,  $(a, c)$ ,  $(b, a)$ ,  $(c, b)$ ,  $(c, a)$  등과 같이 임의의 두 정점들에서 경로가 존재하므로 강하게 연결되어 있다.

41/99

41

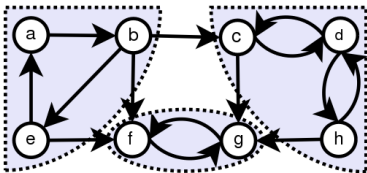
# Connect

## Strongly Connect

- ✓ A directed graph is called strongly connected if there is a path in each direction between each pair of vertices of the graph.
- ✓ That is, a path exists from the first vertex in the pair to the second, and another path exists from the second vertex to the first.
- ✓ In a directed graph  $G$  that may not itself be strongly connected, a pair of vertices  $u$  and  $v$  are said to be strongly connected to each other if there is a path in each direction between them.

## Strongly Connected Component

- ✓ The strongly connected components of an arbitrary directed graph form a partition into subgraphs that are themselves strongly connected.



Graph with strongly connected components marked

[출처] [https://en.wikipedia.org/wiki/Strongly\\_connected\\_component](https://en.wikipedia.org/wiki/Strongly_connected_component)

42/99

42

## 7.2 여러 가지 그래프



43

### 완전 그래프 (complete graph)

#### 정의 7-8 완전 그래프

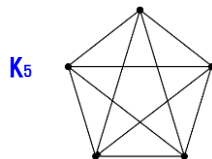
무향 그래프  $G=(V, E)$ 의 모든 정점의 쌍 사이에 간선이 존재하면 무향 그래프  $G$ 를 완전 그래프 complete graph라 한다. Notation :  $K_n$  denotes the complete graph on  $n$  vertices

#### ❖ 완전 그래프의 간선 개수

- $n$ 개의 정점을 갖는 완전 그래프는  $\frac{n(n-1)}{2}$ 개의 간선을 갖는다.

#### 예제 7-8 완전 그래프의 간선의 수

다음과 같은 완전 그래프가 있다. 간선의 수를 구하라.



풀이

정점의 개수가 5개인 완전 그래프이다. 그러므로 간선의 개수는  $\frac{5(5-1)}{2} = 10$  개이다.

- 5개의 정점을 가진 완전 그래프
- 각 정점에서 차수를 구해보면 4차이다.
- $n$ 개의 정점을 갖는 완전 그래프는 모든 정점들이  $(n-1)$  차수를 가진다.
- 그래서 이런 그래프를 4차의 완전 그래프라 한다.

44

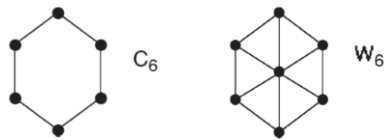
44/99

## Cycle Graph

- Definition : a **cycle graph** has vertices  $\{v_1, v_2, \dots, v_n\}$  and edges  $\{e_1, e_2, \dots, e_n\}$  such that edge  $e_k$  joins vertices  $\{v_k, v_{k+1}\}$  where  $v_{k+1}$  is "mod  $n$ ".
- Notation :  $C_n$  denotes the cycle graph on  $n$  vertices.

## Wheel Graph

- Definition : a **wheel graph** has a cycle graph with an additional hub vertex joined to every other vertex.
- Notation :  $W_n$  denotes the wheel graph on  $n+1$  vertices.



45/99

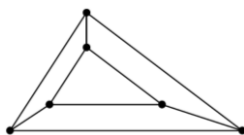
45

## 정규 그래프 (regular graph)

### 정의 7-9 정규 그래프

그래프  $G = (V, E)$ 의 모든 정점의 차수가 같으면 그래프  $G$ 를 **정규 그래프** regular graph라 한다.

### Examples



모든 정점의 차수가 3으로,  
3차 정규 그래프임

1.  $K_n$  is regular of degree  $n-1$
2.  $C_n$  is regular of degree 2
3.  $W_3$  is regular of degree 3



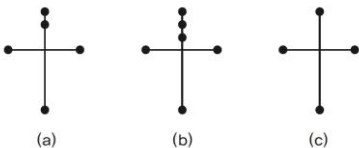
46/99

46

# 동형 그래프(isomorphic graph), 준 동형 그래프(homomorphism graph)

## 정의 7-10 동형 그래프, 준동형 그래프

그래프  $G=(V, E)$ 와  $G^*=(V^*, E^*)$ 에서 함수  $f: V \rightarrow V^*$ 가  $(u, v) \in E \Leftrightarrow (f(u), f(v)) \in E^*$ 를 만족하는 전단사 함수이면, 이때  $G$ 와  $G^*$ 를 동형 그래프 isomorphic graph라 한다. 또한 임의의 그래프  $G$ 에서  $G$ 의 간선 위에 정점을 추가해서 새로운 그래프를 만들 수 있다.  $G'$ 과  $G''$ 이 그래프  $G$ 의 간선 위에 정점을 추가해서 만들어진 그래프라 하면, 이때의  $G'$ ,  $G''$ 을 그래프  $G$ 의 준동형 그래프 homomorphism graph라 한다.



[그림 7-6] 준동형 그래프

- [그림 7-6]의 그래프 (a)와 (b)는 (c)의 준동형 그래프이다.

47/99

47

# 동형 그래프(isomorphic graph), 준 동형 그래프(homomorphism graph)

In graph theory, an **isomorphism of graphs**  $G$  and  $H$  is a **bijection** between the vertex sets of  $G$  and  $H$

$$f: V(G) \rightarrow V(H) \quad \text{전단사}$$

such that any two vertices  $u$  and  $v$  of  $G$  are **adjacent** in  $G$  if and only if  $f(u)$  and  $f(v)$  are adjacent in  $H$ . This kind of bijection is commonly described as "edge-preserving bijection", in accordance with the general notion of **isomorphism** being a structure-preserving bijection

If an **isomorphism** exists between two graphs, then the graphs are called **isomorphic** and denoted as  $G \simeq H$ . In the case when the bijection is a mapping of a graph onto itself, i.e., when  $G$  and  $H$  are one and the same graph, the bijection is called an **automorphism** of  $G$ .

Graph isomorphism is an **equivalence relation** on graphs and as such it partitions the **class** of all graphs into **equivalence classes**. A set of graphs isomorphic to each other is called an **isomorphism class** of graphs.

The two graphs shown below are isomorphic, despite their different looking **drawings**.

| Graph G | Graph H | An isomorphism between G and H                                                                               |
|---------|---------|--------------------------------------------------------------------------------------------------------------|
|         |         | $f(a) = 1$<br>$f(b) = 6$<br>$f(c) = 8$<br>$f(d) = 3$<br>$f(g) = 5$<br>$f(h) = 2$<br>$f(i) = 4$<br>$f(j) = 7$ |

[출처] [https://en.wikipedia.org/wiki/Graph\\_isomorphism](https://en.wikipedia.org/wiki/Graph_isomorphism)

48/99

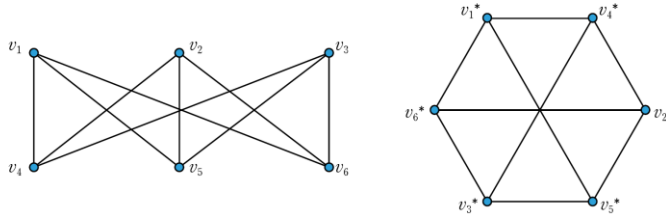
48



## 동형 그래프(isomorphic graph), 준 동형 그래프(homomorphism graph)

### 예제 7-10 동형 그래프 판단

다음 두 그래프가 동형 그래프인지를 판단하라.



#### 풀이

$V = \{v_1, v_2, v_3, v_4, v_5, v_6\}$ 을  $V^* = \{v_1^*, v_2^*, v_3^*, v_4^*, v_5^*, v_6^*\}$ 에 대응시키면  $(u, v) \in E \Leftrightarrow (f(u), f(v)) \in E^*$ 를 모두 만족한다. 그러므로 이 두 그래프는 동형 그래프이다.

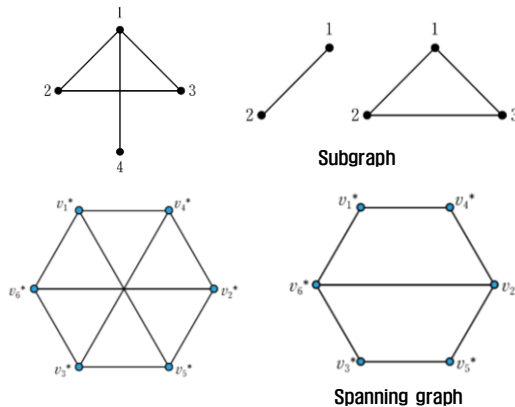
49/99

49

## 부분 그래프, 스패닝 그래프

### 정의 7-11 부분 그래프, 스패닝 그래프

그래프  $G = (V, E)$ 가 있을 때  $V' \subseteq V$ 이고  $E' \subseteq E$ 인 그래프  $G' = (V', E')$ 을  $G$ 의 부분 그래프(subgraph)라고 한다. 특히  $E' \neq E$ 이면 진부분 그래프(proper subgraph)라고 한다. 그리고  $V' = V$ 이고  $E' \subseteq E$ 이면  $G'$ 은  $G$ 의 스패닝 그래프(spanning graph)라고 한다.



50/99

50

## 이분 그래프(bipartite graph), 완전 이분 그래프

### 정의 7-12 이분 그래프, 완전 이분 그래프

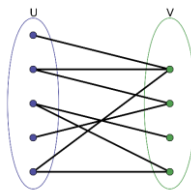
그래프  $G=(V, E)$ 에서 정점의 집합  $V$ 가 두 부분집합  $M$ 과  $N$ 으로 분할되어( $V=M \cup N$ ,  $M \cap N = \emptyset$ ), 두 개의 집합  $M$ 과  $N$  사이에 간선으로 정점들을 연결하는 그래프  $G$ 를 **이분 그래프 bipartite graph**라 한다. 특히  $M$  내의 모든 정점과  $N$  내의 모든 정점 사이에 모두 간선이 존재하면  $G$ 를 **완전 이분 그래프 complete bipartite graph**라 한다. 즉, 완전 그래프이면서 이분 그래프를 만족한다.

#### A bipartite graph

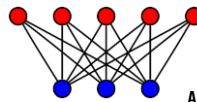
- ✓ the vertex set can be partitioned into two sets,  $W$  and  $X$ , so that no two vertices in  $W$  share a common edge and no two vertices in  $X$  share a common edge.

#### In a complete bipartite graph,

- ✓ the vertex set is the union of two disjoint sets,  $W$  and  $X$ , so that every vertex in  $W$  is adjacent to every vertex in  $X$  but there are no edges within  $W$  or  $X$ .



Example of a bipartite graph without cycles



$K_{5,3}$

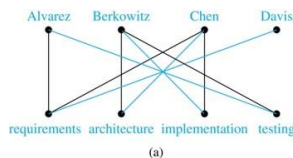
A complete bipartite graph with  $m = 5$  and  $n = 3$

51

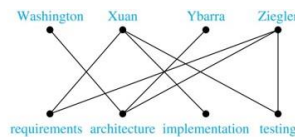
51/99

## Bipartite Graphs and Matchings

- ❑ Bipartite graphs are used to model applications that involve matching the elements of one set to elements in another, for example:
- ❑ **Job assignments** - vertices represent the jobs and the employees, edges link employees with those jobs they have been trained to do. A common goal is to match jobs to employees so that the most jobs are done.



(a)



(b)

- ❑ **Marriage** - vertices represent the men and the women and edges link a man and a woman if they are an acceptable spouse. We may wish to find the largest number of possible marriages.

52

52/99

## 이분 그래프, 완전 이분 그래프

### 예제 7-13 완전 이분 그래프

다음 그래프가 완전 이분 그래프인지 판단하라.



#### 풀이

- (a) 왼쪽에 있는 두 개의 정점과 오른쪽에 있는 세 개의 정점으로 분할하면 왼쪽과 오른쪽 간에 모든 정점들 사이에 간선이 존재하므로 완전 이분 그래프이다.
- (b) 왼쪽에 있는 세 개의 정점과 오른쪽에 있는 세 개의 정점으로 분할하면 왼쪽과 오른쪽 간에 모든 정점들 사이에 간선이 존재하므로 완전 이분 그래프이다.

- 그래프  $G = (V, E)$ 에서 정점의 집합  $V$ 가 두 부분집합  $M$ 과  $N$ 으로 분할되어 있고,  $M$ 의 정점의 개수가  $m$ 이고  $N$ 의 정점의 개수가  $n$ 인 완전 이분 그래프를  $K_{m,n}$ 으로 나타내며,  $K_{m,n}$ 은  $m \times n$ 의 간선을 가진다.
- [예제 7-13]의 그래프는  $K_{2,3}, K_{3,3}$  그래프이다.

53/99

53

## 희소 그래프, 밀집 그래프

### 정의 7-13 희소 그래프, 밀집 그래프

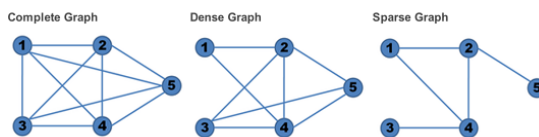
그래프  $G = (V, E)$ 의 간선의 개수가  $|V|^2$ 보다 훨씬 작은 그래프를 **희소 sparse 그래프**라 하고 주로 인접 리스트를 사용해서 표현한다. **밀집 dense 그래프**는 간선의 개수가  $|V|^2$ 과 비례하는 그래프를 말하고, 이는 주로 인접 행렬을 사용해서 표현한다.

[Wikipedia]

- A **dense graph** is a graph in which the number of edges is close to the maximal number of edges. The opposite, a graph with only a few edges, is a **sparse graph**.
- The **graph density** of simple graphs is defined to be the ratio of the number of edges  $|E|$  with respect to the maximum possible edges

The maximum number of edges of an undirected simple graphs  $\binom{|V|}{2} = |V|(|V| - 1)/2$   
The maximal density is 1 (Complete graphs)

|               | Undirected simple graphs                                     | Directed simple graphs                                       |
|---------------|--------------------------------------------------------------|--------------------------------------------------------------|
| Graph density | $D = \frac{ E }{\binom{ V }{2}} = \frac{2 E }{ V ( V  - 1)}$ | $D = \frac{ E }{2\binom{ V }{2}} = \frac{ E }{ V ( V  - 1)}$ |



[그림출처]  
[https://www.alberton.info/graphs\\_in\\_the\\_database\\_sql\\_meets\\_social\\_networks.html](https://www.alberton.info/graphs_in_the_database_sql_meets_social_networks.html)

54/99

54

# 가중치 그래프

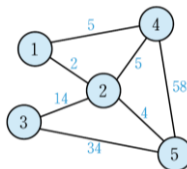
### 정의 7-14 가중치 그래프

가중치 <sup>weighted</sup> 그래프는 그래프  $G=(V, E)$ 의 간선에 음수가 아닌 가중치를 할당한 그래프이다.

- 가중치는 두 정점 사이의 거리, 시간, 비용(cost) 등을 나타낸다.
- 전력선 연결, 파이프라인 연결, 도로망 건설, 컴퓨터 네트워크 등에 응용된다.

### 예제 7-14 가중치 그래프

다음과 같은 가중치 그래프를 보고 정점 1로부터 정점 5로 가는 가장 작은 가중치를 가지는 경로를 구하라.



### 풀이

정점 1에서 정점 5로 가는 가장 작은 가중치를 가지는 경로는 1, 2, 5로서 가중치 6을 가진다.

55/99

55

# 트리

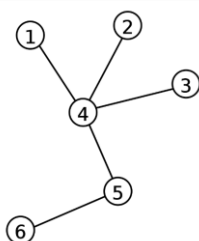
## 8장 트리 학습

### 정의 7-15 트리

순환을 갖지 않고 루트 노드 <sup>root node</sup>라고 하는 한 개의 특별한 정점을 갖는 연결 그래프를 트리 <sup>tree</sup> 혹은 수형도라 한다.

[Wikipedia]

- In graph theory, a **tree** is an **undirected graph** in which any two vertices are connected by exactly one path, or equivalently a connected acyclic undirected graph



A labeled tree with 6 vertices and 5 edges.

Vertices :  $V$   
Edges :  $V-1$

56/99

56

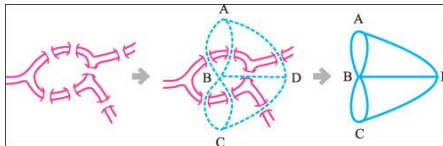
## 오일러 루프 (Euler loop), 오일러 그래프 (Euler graph)

### 정의 7-16 오일러 루프, 오일러 그래프

그래프  $G$ 가 있다고 할 때  $G$ 의 한 정점에서 시작하여  $G$ 의 모든 간선을 딱 한 번만 거쳐서 다시 시작한 정점으로 돌아오는 루프를 **오일러 루프** Euler loop라고 하고, 이러한 루프를 갖는 그래프를 **오일러 그래프** Euler graph라고 한다.

❖ **Eulerian Path** is a path in graph that visits every edge exactly once. **Eulerian Circuit** is an Eulerian Path which starts and ends on the same vertex.

- 그래프 이론을 제시한 코니히스베르크의 다리 건너기 문제는 오일러 그래프에 대한 문제이다.
- 오일러 그래프는 한붓그리기로 잘 알려져 있다. 그래프의 한 정점에서 시작하여 붓을 떼지 않고 모든 간선을 딱 한 번씩만 지나서 다시 자기 자신으로 돌아오는 루프이다.



[그림출처] <https://steemit.com/kr-science/@rubymaker/tb24i-rubymaker>

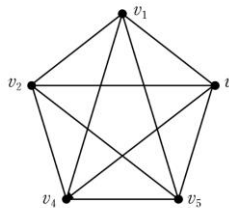
57/99

57

## 오일러 루프, 오일러 그래프

### 예제 7-15 오일러 루프

다음 그래프에서  $v_1v_2v_4v_5v_3v_2v_5v_1v_4v_3v_1$ 이 오일러 루프인지 판단하라.



### 풀이

오일러 루프가 되기 위해서는 시작점과 종점이 같은지를 확인하면 된다.  $v_1$ 로 같으므로 루프이다. 그리고 모든 간선들을 단 한 번만 지나므로 오일러 루프가 맞다.

58/99

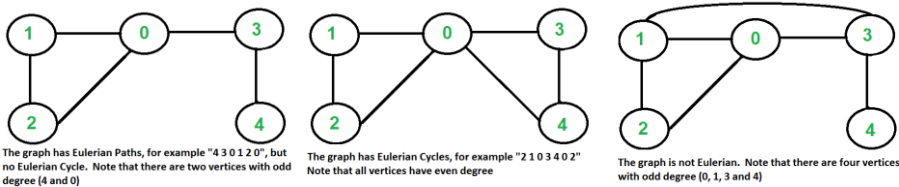
58

# 오일러 루프, 오일러 그래프

정리 7-1 오일러 그래프가 될 필요충분조건

연결 그래프  $G$ 가 오일러 그래프가 될 필요충분조건은  $G$ 의 모든 정점이 짝수의 차수를 갖는다는 것이다.

- [예제 7-15]의 그래프는 오일러 그래프이다.
- 왜냐하면 모든 정점들의 차수가 짝수이기 때문이다.



[그림출처] <https://www.geeksforgeeks.org/eulerian-path-and-circuit/>

59/99

# 해밀턴 그래프 (Hamilton graph)

정의 7-17 해밀턴 그래프

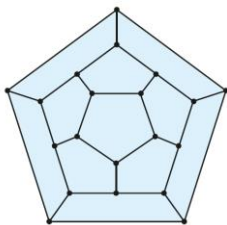
그래프  $G$ 가 있다고 할 때  $G$ 의 어떤 정점에서 시작하여 모든 정점들을 단 한 번만 지나서 다시 시작 정점까지 돌아오는 순환을 해밀턴 Hamilton 순환이라 하고, 이런 순환을 갖는 그래프를 해밀턴 Hamilton 그래프라 한다.

오일러 경로, 오일러 순환과는 다르게 해밀턴 경로, 해밀턴 순환은 같은 모서리를 몇 번을 지나든 상관없이 꼭짓점들만 한 번씩 지나면 됨

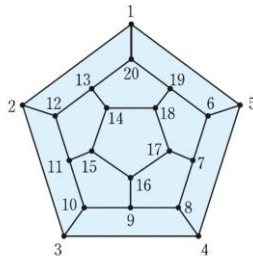
60/99

# 해밀턴 그래프

- 아일랜드의 대학자인 해밀턴(W. R. Hamilton, 1805~1865)은 12면체 모양의 수수께끼 하나를 제시했다.
- 각 정점에 도시 이름을 적고, 어떤 도시에서 출발하여, 간선들을 따라서 한 도시를 한 번씩만 방문한 후 최초의 출발 도시로 돌아오는 것이다.
- 12면체의 평면 버전은 [그림 7-7]과 같다.
- 해밀턴의 수수께끼는 [그림 7-7]에서 순환을 찾을 수 있다면 해결된다. 다만, 각 정점은 단 한 번만 포함되어야 한다.
- 해밀턴 순환은 [그림 7-8]에서 연속적으로 순서가 붙은 정점들로 이루어져 있다.



[그림 7-7] 12면체 평면 버전



[그림 7-8] 해밀턴 순환

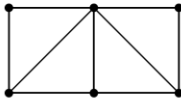
61/99

61

# 해밀턴 그래프

## 예제 7-16 오일러 그래프와 해밀턴 그래프 판단

다음 그래프가 오일러 그래프인지 해밀턴 그래프인지를 판단하라.



### 풀이

오일러 그래프가 되기 위해서는 모든 정점들의 차수가 짝수여야 한다. 그런데 모든 정점의 차수가 짝수가 아니므로 오일러 그래프는 아니다. 주어진 그래프는 다음과 같은 경로를 따라서 모든 정점을 지나 다시 시작점으로 올 수 있다. 그러므로 해밀턴 그래프이다.



즉, 해밀턴 그래프이지만 오일러 그래프는 아니다.

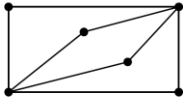
62/99

62

# 해밀턴 그래프

## 예제 7-17 오일러 그래프와 해밀턴 그래프 판단

다음 그래프가 오일러 그래프인지 해밀턴 그래프인지를 판단하라.



### 풀이

모든 정점들의 차수가 짝수이므로 오일러 그래프이다. 모든 정점들을 단 한 번만 지나서 다시 자기 자신의 정점으로 돌아오는 해밀턴 순환이 존재하지 않으므로 해밀턴 그래프는 아니다.

63/99

63

# 판매원 탐방 문제

## 해밀턴 순환과 관련 있는 판매원 방문 문제

- 판매원 탐방 문제(traveling salesperson problem)는 그래프에서 해밀턴 순환을 찾는 문제와 관련이 있다.
- 이 문제는 가중치 그래프  $G$ 가 주어지면  $G$  안에서 최소 길이를 가지는 해밀턴 순환을 찾는 문제이다.
- 가중치 그래프에서 도시를 정점으로 간선에 가중치를 거리로 주는 경우에, 판매원 탐방 문제는 판매원이 어느 도시를 출발하여 각 도시를 한 번만 방문하여 출발지로 다시 돌아오는 최소 거리의 순환을 찾는 문제이다.
- 이 문제는 완전 그래프에서 전체 가중치가 가장 적은 해밀턴 순환을 찾는 것과 같다.

64/99

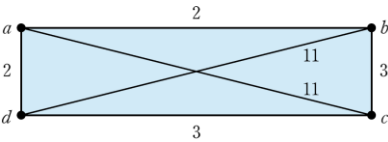
64



# 판매원 탐방 문제

## 예제 7-18 판매원 탐방 문제

다음과 같은 가중치 그래프에서 판매원 탐방 문제를 해결하라.



### 풀이

그래프  $G$ 에서 해밀턴 순환은 여러 개가 존재하지만 가중치의 합이 최소가 되는 최소 길이의 해밀턴 순환은  $(a, b, c, d, a)$ 이다.

65/99

65

# 그래프의 탐색

## 그래프의 탐색

- 무향 그래프  $G = (V, E)$ 와  $V$ 에 있는 정점  $v$ 가 주어져 있을 때, 이 정점  $v$ 로부터 시작하여 도달할 수 있는  $G$ 의 모든 정점을 탐색하는 것은 매우 중요한 일이다.
- 그래프의 각 정점을 한번씩만 방문하는 방법
  - ✓ 깊이우선 탐색(depth first search ; DFS)
  - ✓ 너비우선 탐색(breadth first search ; BFS)

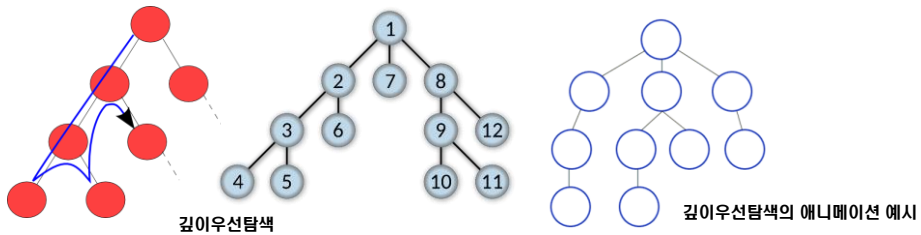
66/99

66

## 깊이우선 탐색 (DFS)

### 깊이우선 탐색

- 먼저 주어진 정점  $v$ 를 출발점으로 하여 이를 방문한다.
- 다음  $v$ 에 인접하고 아직 방문하지 않은 정점  $w$ 를 선택하여  $w$ 를 출발점으로 해서 다시 깊이 우선 탐색을 시작한다.
- 이것을 모든 정점이 한 번씩 방문 될 때까지 반복한다.
- 인접한 모든 정점들이 이미 방문한 정점  $v$ 인 경우는 가장 최근에 방문했던 정점에서, 방문하지 않은 정점  $w$ 를 가진 정점을 선택하여 정점  $w$ 로부터 다시 깊이우선 탐색을 시작한다.
- 방문한 어떤 정점으로부터 방문하지 않은 정점에 도달할 수 없을 때 탐색이 끝난다.



67/99

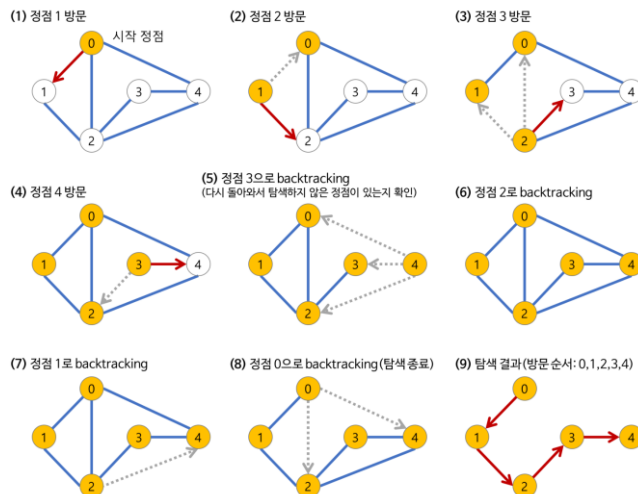
67

## 깊이우선 탐색 (DFS)

### 깊이우선 탐색

#### Pseudocode

```
void DFS(v)
int v;
{
    int w;
    extern int VISITED[];
    VISITED[v]=1;
    while(v에 인접한 모든 노드 w)
        if( !VISITED[w] )
            DFS(w)
}
```



[그림출처] <https://gmlwjd9405.github.io/2018/08/14/algorithm-dfs.html>

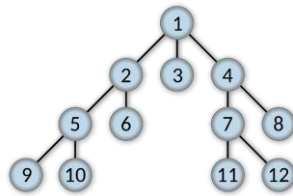
68/99

68

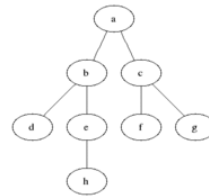
## 너비우선 탐색 (BFS)

### 너비우선 탐색

- 먼저 주어진 정점  $v$ 를 출발점으로 하여 이를 방문한다.
- 그리고  $v$ 에 인접한 정점  $w$ 들을 먼저 모두 방문하고 그 다음으로  $w$ 에 인접하고 아직 방문하지 않은 정점들을 모두 방문한다.
- 위의 과정을 반복하여 더 이상 방문할 노드가 없을 때까지 계속하여 방문한다.



너비우선탐색



너비우선탐색의 애니메이션 예시

69

69/99

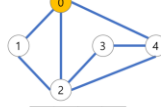
## 너비우선 탐색 (BFS)

### 너비우선 탐색

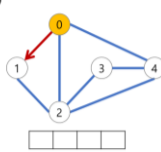
#### Pseudocode

```
void BFS(v)
int v;
{
    int w;
    extern struct queue ·q;
    VISITED[v]=1;
    InitializeQueue(q);
    EnQueue(q, v);
    while( !q.empty() ){
        v=DeQueue(q);
        while(v에 인접한 모든 노드 w){
            if( !VISITED[w] ):
                EnQueue(q,w);
                VISITED[w]=1;
        }
    }
}
```

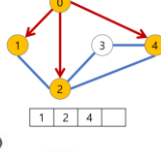
(1) 시작 정점 방문



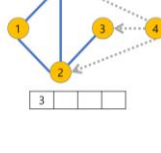
(2) 큐(Queue)



(3)



(4)



(5)



(6)



(7)



(8)



(9)



(10) 탐색 결과(방문 순서: 0,1,2,4,3)



[그림출처]

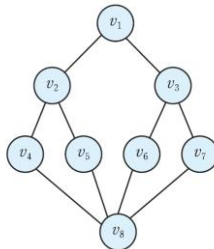
<https://gmlwjd9405.github.io/2018/08/15/algorithm-bfs.html>

70

70/99

## 너비우선 탐색 (BFS)

 다음의 그래프를 깊이우선 탐색과 너비우선 탐색을 해보자.



[그림 7-9] 그래프

- $v_1$ 을 출발점으로 하여 깊이우선 탐색에 의해서 방문하면  $v_1, v_2, v_4, v_8, v_5, v_6, v_3, v_7$ 이 된다.
- $v_1$ 을 출발점으로 하여 너비우선 탐색에 의해서 방문하면  $v_1, v_2, v_3, v_4, v_5, v_6, v_7, v_8$ 이 된다.

71/99

71

## 최단 경로 (Dijkstra's algorithm)

 최단 경로

- 가중치 그래프를 이용한 두 정점 사이에 최단 경로(shortest path)를 찾는 문제를 생각해보자.
- 두 정점 사이에 최단 경로의 길이는 가중치들의 합 중에서 가장 작은 값을 가지는 경로를 말한다.
- 정점  $u$ 에서 정점  $v$  사이에 최단 경로를 두 정점 사이의 거리라고 한다.
- 만일 어떤 두 정점 사이에 최단 경로가 없다면 이때의 거리는 무한대( $\infty$ )라고 가정한다.
- 그러면 최단 경로를 어떻게 하면 찾을 수 있을까?  
그것에 대한 명백한 해답은 모든 경로에 대해서 거리를 계산하는 것이다.
- 그러나 이 방법은  $n$  개의 정점들이 있을 때 정점  $a$ 로부터 정점  $b$ 로의 경로는  $(n-2)!$  이상이 존재하므로 만일  $n = 30$ 일 경우에 경로를 컴퓨터로 계산한다고 하더라도 엄청난 시간을 요구할 것이다. 그러므로 모든 경우의 경로를 구해서 계산한다는 것은 무리이다.
- 이를 해결하는 알고리즘의 하나가 Dijkstra의 알고리즘이다
- Dijkstra는 네덜란드 사람으로 이론 물리학자이며 1972년 ACM으로부터 그 유명한 튜링(Alan Turing) 상을 수상했다. 튜링 상은 컴퓨터 분야에서의 노벨상이라고 불린다.

72/99

72

# 최단 경로 (Dijkstra's algorithm)

## Dijkstra 최단 경로 알고리즘

- 하나의 시작 정점에서 다른 정점까지의 최단 경로를 구함
- 단일점에서의 최단 경로One to All Shortest Path 알고리즘 중 가장 많이 사용.
- 무방향 그래프나 방향 그래프에 모두 적용 가능

- ❶

경로 길이를 저장할 배열 distance 준비 : 시작 정점으로부터 각 정점에 이르는 경로의 길이를 저장하기 위한 배열 distance를 ∞(무한대)로 초기화한다.
- ❷

시작 정점 초기화 : 시작 정점의 distance를 0으로 초기화하고 집합 S에 추가한다.
- ❸

최단 거리 수정 : 집합 S에 속하지 않은 정점 중에서 distance가 최소인 정점 u를 찾아 집합 S에 추가한다. 새로운 정점 u가 추가되면, u에 인접하고 집합 S에 포함되지 않은 정점 w의 distance값을 다음 식에 따라 수정한다. 집합 S에 모든 정점이 추가될 때까지 ❸을 반복한다.

$$distance[w] \leftarrow \min (distance[w], distance[u]+weight[u][w])$$

그림 8-28 다익스트라 최단 경로 알고리즘에서 최단 경로 찾기

[자료출처: IT CookBook, C로 배우는 쉬운 자료구조(개정3판), 2016

73/99

# 최단 경로 (Dijkstra's algorithm)

## Dijkstra 최단 경로 알고리즘

- 시작 정점 v에서 정점u까지의 최단 경로 distance[u]를 구해 정점 u가 집합 S에 추가되면 새로 추가된 정점 u에 의해 단축되는 경로가 있는지를 확인함.
- 즉, 현재 알고 있는 distance[w]를 새로 추가된 정점 u를 거쳐서 가는 distance[u]+weight[u][w]를 계산한 경로와 비교해 경로가 단축되면 distance[w]를 단축된 경로값으로 수정함으로써 최단 경로를 찾음

$$distance[w] \leftarrow \min (distance[w], distance[u]+weight[u][w])$$

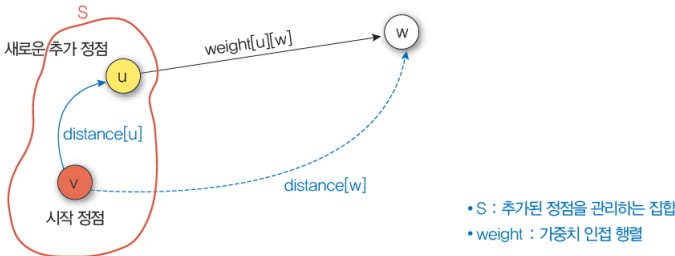


그림 8-27 다익스트라 최단 경로 알고리즘에서 최단 경로 구하기의 기본 개념

[자료출처: IT CookBook, C로 배우는 쉬운 자료구조(개정3판), 2016

74/99

최단 경로 (Dijkstra's algorithm)

가중치 인접 행렬 (Weight Adjacent Matrix)

- 최단 경로를 구하려는 가중치 그래프의 가중치를 저장
- 그래프에서 사용한 인접 행렬과 같은 형태의 2차원 배열, 두 정점 사이에 간선이 없으면 0이 아니라 ∞[무한대]값이 저장되어 있다고 가정.

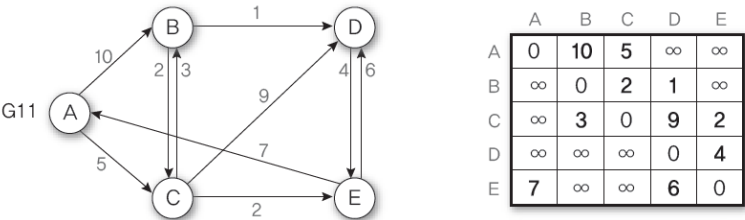


그림 8-26 가중치 방향 그래프와 가중치 인접 행렬의 예

[자료출처: IT CookBook, C로 배우는 쉬운 자료구조(개정3판), 2016

75/99

최단 경로 (Dijkstra's algorithm)

Dijkstra 최단 경로 알고리즘

알고리즘 8-3 다익스트라 최단 경로

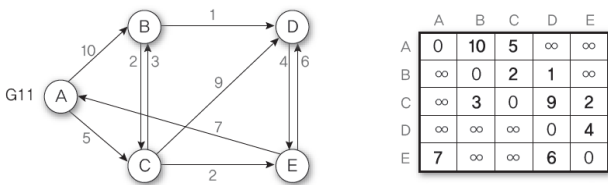
```
Dijkstra_shortestPath(G,v)
S ← {v};
for (i ← 0; i ∈ V(G); i ← i+1) do
    distance[i] ← weight[v][i];
for(i ← 0; S != V(G); i ← i+1 ) do { // 집합 S와 정점의 집합 V(G)가 같지 않은 동안
    u ← S에 속하지 않은 정점 중에서 distance[]가 최소인 정점;
    S ← S ∪ {u};
    for (w ← 0; w ∈ V(G); w ← w+1) do
        if(z ∉ S) then
            distance[w] ← min(distance[w], distance[u]+weight[u][w]);
    }
end Dijkstra_shortestPath()
```

[자료출처: IT CookBook, C로 배우는 쉬운 자료구조(개정3판), 2016

76/99

# 최단 경로 (Dijkstra's algorithm)

- 아래 그래프 G11에 Dijkstra's algorithm 적용하여 최단 경로를 구하기
- 시작 정점은 A, 각 정점 옆에 붙은 숫자는 시작 정점 A에서 각 정점까지 도달하는 거리를 의미. 집합 S에 포함되는 추가 경로 거리는 노란색, 집합 S의 정점은 빨간색, 현재 단계에서 새로 추가된 정점과 그로 인해 수정된 distance값은 파란색으로 표시



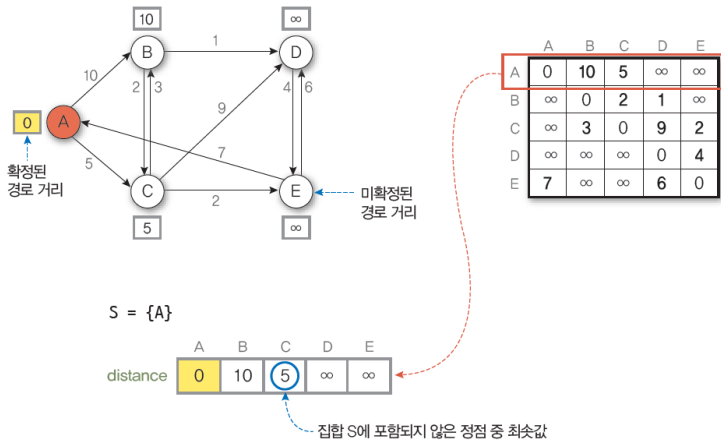
[자료출처: IT CookBook, C로 배우는 쉬운 자료구조(개정3판), 2016]

77/99

77

# 최단 경로 (Dijkstra's algorithm)

- 초기 상태
- 시작 정점 A를 집합 S의 초기값으로 설정, 시작 정점 A에서의 인접 정점에 대한 가중치값을 인접 행렬 weight에서 구해 배열 distance의 초기값으로 설정



[자료출처: IT CookBook, C로 배우는 쉬운 자료구조(개정3판), 2016]

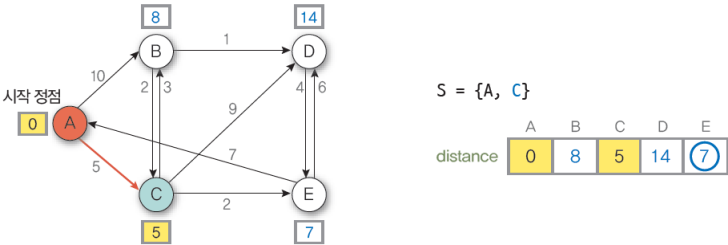
78/99

78

# 최단 경로 (Dijkstra's algorithm)

- ① 시작 정점 A의 진출 간선과 연결된 인접 정점 B와 C 중에서 최단 경로인 C를 집합 S에 추가.  
 새로 추가된 정점 C에 의해 단축되는 경로가 있는지 확인하여 배열 distance값을 수정

$$\begin{aligned} \text{distance}[B] &\leftarrow \min(\text{distance}[B], \text{distance}[C] + \text{weight}[C][B]) = \min(10, 5 + 3) = 8 \\ \text{distance}[D] &\leftarrow \min(\text{distance}[D], \text{distance}[C] + \text{weight}[C][D]) = \min(\infty, 5 + 9) = 14 \\ \text{distance}[E] &\leftarrow \min(\text{distance}[E], \text{distance}[C] + \text{weight}[C][E]) = \min(\infty, 5 + 2) = 7 \end{aligned}$$



[자료출처: IT CookBook, C로 배우는 쉬운 자료구조(개정3판), 2016]

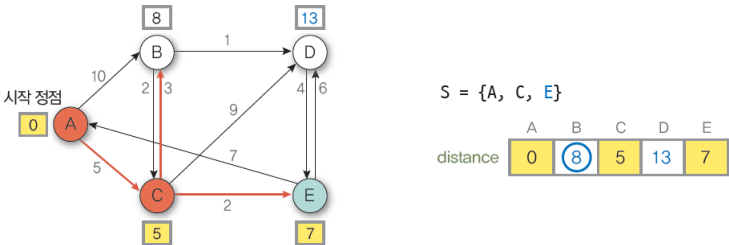
79/99

79

# 최단 경로 (Dijkstra's algorithm)

- ② 집합 S에 포함되지 않은 정점 중에서 최단 경로인 E를 집합 S에 추가.  
 새로 추가된 정점 E에 의해 단축되는 경로가 있는지 확인하여 배열 distance값을 수정

$$\text{distance}[D] \leftarrow \min(\text{distance}[D], \text{distance}[E] + \text{weight}[E][D]) = \min(14, 7 + 6) = 13$$



[자료출처: IT CookBook, C로 배우는 쉬운 자료구조(개정3판), 2016]

80/99

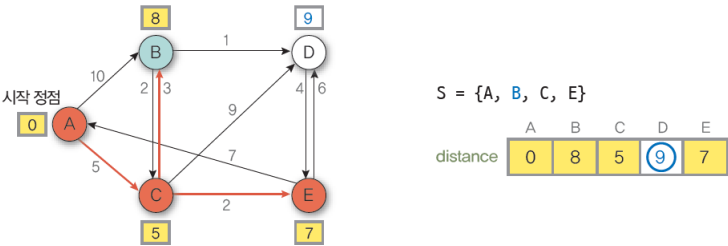
80



# 최단 경로 (Dijkstra's algorithm)

- ③ 집합 S에 포함되지 않은 정점 중에서 최단 경로인 B를 집합 S에 추가.  
 새로 추가된 정점 B에 의해 단축되는 경로가 있는지 확인하여 배열 distance값을 수정

$$\text{distance}[D] \leftarrow \min(\text{distance}[D], \text{distance}[B] + \text{weight}[B][D]) = \min(13, 8 + 1) = 9$$



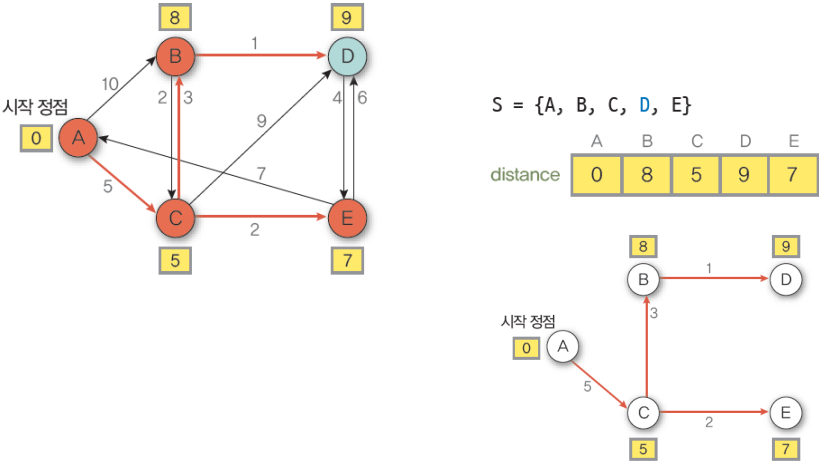
[자료출처; IT CookBook, C로 배우는 쉬운 자료구조(개정3판), 2016]

81/99

81

# 최단 경로 (Dijkstra's algorithm)

- ④ 집합 S에 포함되지 않은 정점 중에서 최단 경로인 D를 집합 S에 추가.  
 이로써 모든 정점이 집합 S에 추가되었으므로 최단 경로가 완성됨



[자료출처; IT CookBook, C로 배우는 쉬운 자료구조(개정3판), 2016]

그림 8-29 그래프 G11의 최단 경로 트리

82/99

82

## 7.3 평면 그래프

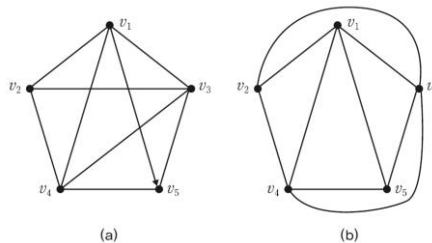


83

### 교점, 교선

#### 교점(cross over), 교선 (cross line)

- 평면상에 그래프를 그릴 때, 정점 이외의 점에서 간선을 서로 교차해서 그릴 수 있어야 편리하다.
- 이때 교차되는 점을 **교점**이라 하고 교차되는 간선을 **교선**이라고 한다.
- [그림 7-11(a)]에서 간선  $(v_1, v_4)$ 는 간선  $(v_2, v_3)$ 과 교차하면 간선  $(v_1, v_5)$ 는 간선  $(v_2, v_3)$ ,  $(v_3, v_4)$ 와 교차한다.



[그림 7-11] 평면 그래프

84

84/99

## 평면 그래프

### 정의 7-18 평면 그래프

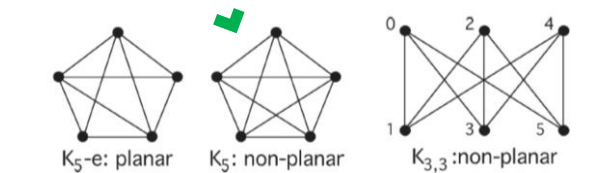
어떤 교점도 갖지 않고 평면 위에 그릴 수 있는 그래프를 **평면 그래프** planar graph라 한다.

- [그림 7-11(a)]는 평면 그래프이다.
- 왜냐하면 [그림 7-11(b)]와 같이 교점이 없이 그래프를 그릴 수 있기 때문이다.
- 평면 그래프를 평면상에 그린 것을 도형(map)이라 한다.

**Definition :** a graph is **planar** if it can be drawn without edge crossing in the plane

**Imbedding problem :** Given a graph  $G$  and a surface  $S$ , is it possible to draw  $G$  on  $S$  without any edge-crossings?

**Planarity problem :** imbedding problem where  $S$  is the sphere or the plane.



85/99

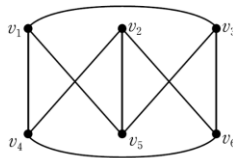
85

## 평면 그래프

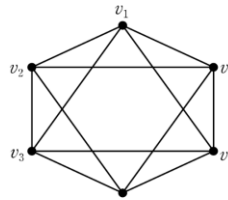
### 예제 7-19 평면 그래프 그리기

다음의 그래프를 평면 그래프로 바꿔라.

(a)

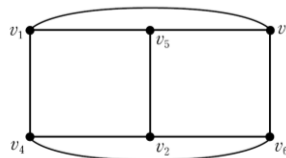


(b)

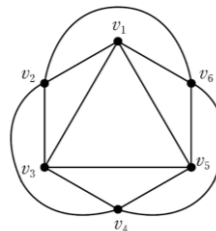


**풀이**

(a)  $v_2$ 와  $v_5$ 의 위치만 바꾸기만 하면 된다.



(b)  $v_2$ 와  $v_4$ 를 연결하는 간선과  $v_1$ 과  $v_5$ 를 연결하는 간선을 다음과 같이 그리면 된다.



86/99

86

# 다각형 그래프

## 정의 7-19 다각형 그래프

하나의 순환으로 된 그래프를 **다각형** polygon이라 한다. 그리고 **다각형 그래프** polygonal graph는 다음과 같이 반복적으로 정의할 수 있는 그래프이다.

- (1) 다각형은 다각형 그래프이다.
- (2)  $G=(V, E)$ 는 다각형 그래프라 하고,  $\alpha = v_iv_{i1}v_{i2}\cdots v_{in-1}v_j(v_i, v_j\in V, v_{ik}\notin V(k=1, 2, \cdots, n-1))$ 은 길이가  $n(\geq 1)$ 인 고유 경로라 하자. 이때  $G$ 와  $\alpha$ 로 이루어진 그래프  $(V', E')$ 도 다각형 그래프이다. 여기서

$$V' = V \cup \{v_{i1}, v_{i2}, \cdots, v_{in-1}\}$$

$$E' = E \cup \{\{v_i, v_{i1}\}, \{v_{i1}, v_{i2}\}, \cdots, \{v_{in-1}, v_j\}\}$$

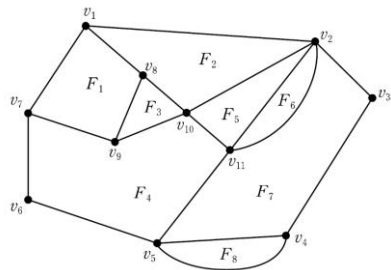
87/99

87

# 다각형 그래프

## 다각형 그래프, 면(face), G의 최대 순환(maximal cycle)

- [정의 7-19]로부터 다각형 그래프는 평면을 여러 영역으로 분할하여 각 영역이 다각형에 의해서 경계를 이루는 평면 그래프라는 것을 알 수 있다.
- 다각형 그래프  $G$ 에 의해서 정의되는 영역들을  $G$ 의 면(face)이라 하고 [[그림 7-12]에서  $F_1, F_2, \cdots, F_8$ ]이라 하고  $G$ 의 모든 면을 포함하고 있는 다각형을  $G$ 의 최대 순환(maximal cycle)라 한다. [[그림 7-12]에서  $v_1v_2v_3v_4v_5v_6v_7v_1$ ].



[그림 7-12] 다각형 그래프

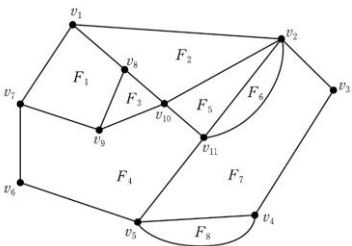
88/99

88

# 다각형 그래프

## 면 $r$ 의 차수(degree)

- 면  $r$ 의 차수(degree)는  $r$ 의 경계를 이루는 경로의 길이이고  $\deg(r)$ 로 나타낸다.
- 어떤 간선이 두 면간의 경계를 이루든지 한 면 내에 포함되어 있든지 관계없이 도형의 모든 면의 경계의 경로를 고려하면 그 간선은 두 번 나타난다.
- 그러므로 한 도형에서 각 영역의 차수의 총합은 그 도형의 간선의 개수의 두 배가된다.
- [그림 7-12]에서  $\deg(F_1) = 4$ ,  $\deg(F_2) = 4$ 이다.



[그림 7-12] 다각형 그래프

89/99

89

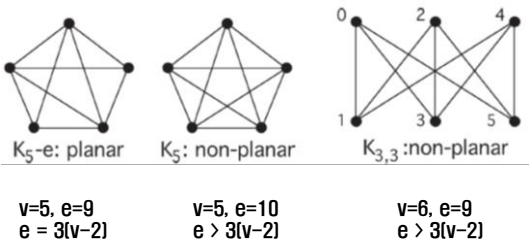
# 다각형 그래프

## 정리 7-2 오일러의 정리

어떤 연결된 도형의 정점의 수를  $v$  간선의 수를  $e$ , 면의 수를  $f$  라 할 때 다음을 만족한다.

$$v-e+f=2$$

- 정리 : 연결된 planar graph의 정점 수  $v$ , 간선의 수  $e$ 라 할 때,  $v \geq 3$ 이면 
$$e \leq 3(v-2)$$



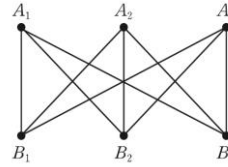
90/99

90

## 비평면 그래프

### 비평면 그래프(nonplanar graph)

- 평면 그래프가 아닌 그래프를 비평면 그래프라 한다.
- [그림 7-13]은 대표적인 비평면 그래프이다.



[그림 7-13] 비평면 그래프

- [그림 7-13]에서  $K_{3,3}$ 은 6개의 정점과 9개의 간선을 가지므로  $K_{3,3}$ 이 평면 그래프라면 오일러의 식에 의해 5개의 면을 가져야 한다.
- $A_1, A_2, A_3$ 의 정점은 서로 연결하지 않고  $B_1, B_2, B_3$ 의 정점을 서로 연결되지 않기 때문에 각 면의 차수가 4이상임을 알 수 있다.
- 따라서 각 면의 차수의 총합이 20이상이어야 하므로 10개 이상의 간선을 가져야 하는데  $K_{3,3}$ 의 간선을 9개라는 사실과 모순된다. 그러므로  $K_{3,3}$ 은 비평면 그래프이다.
- 그래프가 평면 그래프인가 비평면 그래프인가를 결정하는 방법은 많은 수학자들에 의해 연구되어 왔는데 1930년 폴란드의 수학자 쿠라토프스키(K. Kuratowski)에 의해 해결되었다.

91/99

91

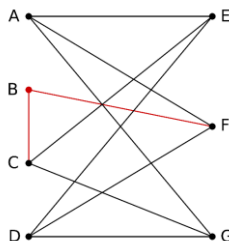
## 비평면 그래프

### 정리 7-3 쿠라토프스키의 정리 Kuratowski's theorem:

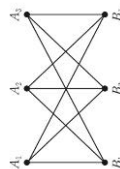
임의의 그래프가 비평면 그래프일 필요충분조건은 그 그래프가  $K_{3,3}$ 과 같이 준동형인 부분 그래프를 포함하는 것이다.

A finite graph is **planar** if and only if it does **not** contain a subgraph that is a subdivision of the complete graph  $K_5$  or the complete bipartite graph  $K_{3,3}$  (utility graph).

A subdivision of a graph results from inserting vertices into edges (for example, changing an edge  $\bullet \text{---} \bullet$  to  $\bullet \text{---} \bullet \text{---} \bullet$ ) zero or more times.



An example of a graph with no  $K_5$  or  $K_{3,3}$  subgraph. However, it contains a subdivision of  $K_{3,3}$  and is therefore non-planar.



92/99

92

## 7. 4 정점의 착색

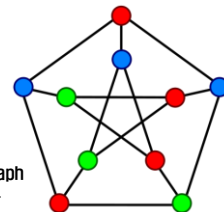


93

### $n$ -색 가능, 착색 수

#### $n$ -색 가능, 착색 수

- 그래프  $G$ 의 정점 착색(vertex coloring) 또는 그래프 착색(graph coloring)은 인접된 정점이 서로 다른 색을 가지도록  $G$ 의 정점들에 색을 할당하는 것이다.
- $n$ 가지의 색으로  $G$ 의 착색이 가능하면  $G$ 는  $n$ -색 가능( $n$ -colorable)이라고 하며,  $G$ 를 착색하는 데 필요한 최소의 색의 가지 수를  $G$ 의 착색수(chromatic number)라 하고  $\chi(G)$ 로 표시한다.
- 이와 같은 정점의 착색은 레지스터 할당 문제 등에 응용된다.



A proper vertex coloring of the Petersen graph with 3 colors, the minimum number possible.

#### 정의 7-20 고유 착색

두 개의 서로 다른 인접한 정점이 같은 색을 갖지 않으면 이러한 착색을 고유 착색(proper coloring)이라 한다.

- 그래프  $G$ 를 착색할 때 웰치-포웰(Welch-Powell)의 알고리즘을 적용할 수 있다.

94/99

94

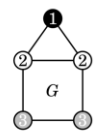
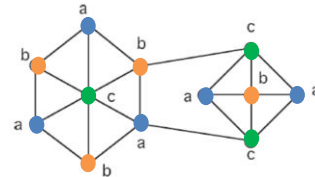
## n-색 가능, 착색 수

### Chromatic number

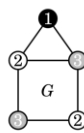
- Definition : a **chromatic number** of a graph  $G$  is the minimum number of colors for which the graph is colorable:

$$\chi(G) = \min\{n \in \mathbb{Z}^+ \text{ such that } G \text{ is } n\text{-colorable}\}$$

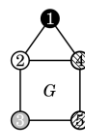
- Definition : a graph is **n-chromatic** if  $\chi(G)=n$
- Example : a graph that contains  $K_3$  requires at least 3 colors.



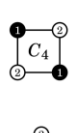
3-coloring  
(not proper)



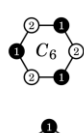
3-coloring  
(proper)



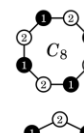
5-coloring  
(proper)



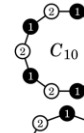
$C_4$



$C_6$



$C_8$



$C_{10}$



$C_3$



$C_5$



$C_7$



$C_9$

A coloring of a graph is said to be a **proper coloring** if no two adjacent vertices have the same color, that is, if the endpoints of each edge have different colors

For even cycles,  $\chi(C_n)=2$ . For odd cycles,  $\chi(C_n)=3$ .

95/99

95

## Welch-Powell 알고리즘

### Welch-Powell의 알고리즘

- In graph theory, **vertex coloring** is a way of labelling each individual vertex such that no two adjacent vertex have same color.
- But we need to find out the number of colors we need to satisfy the given condition. It is not desirable to have a large variety of colors or labels.
- Welsh Powell algorithm that **gives the minimum colors** we need. This algorithm is also used to **find the chromatic number of a graph**. This is an **iterative greedy approach**.
- Welch-Powell algorithm consists of following steps:
  - 1) Find the degree of each vertex
  - 2) List the vertices in order of descending degrees.
  - 3) **Color the first vertex with color 1.**
  - 4) **Move down the list and color all the vertices not connected to the colored vertex, with the same color.**
  - 5) Repeat step 4 on all uncolored vertices with a new color, in descending order of degrees until all the vertices are colored

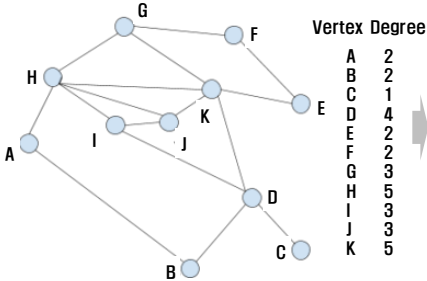
96/99

96



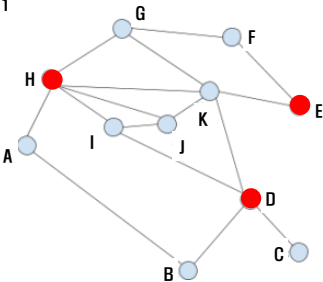
# Welch-Powell 알고리즘

## Welch-Powell의 알고리즘



1) First, order the list in descending order of degrees. Incase of tie, we can randomly choose any ways to break it.

So, the new order will be :  
H, K, D, G, I, J, A, B, E, F, C



2) Now, Following Welsh Powell Graph coloring Algorithm.

- H – color Red
- K – don’ t color Red, as it connects to H
- D – color Red
- G – don’ t color Red, as it connects to H
- I – don’ t color Red, as it connects to H
- J – don’ t color Red, as it connects to H
- A – don’ t color Red, as it connects to H
- B – don’ t color Red, as it connects to D
- E – color Red
- F – don’ t color Red, as it connects to E
- C – don’ t color Red, as it connects to D

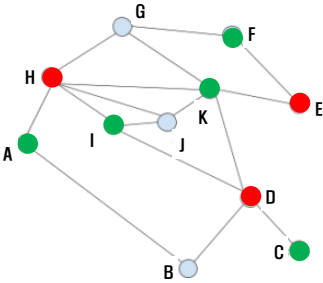
97/99

# Welch-Powell 알고리즘

3) Ignoring the vertices already colored, we are left with : K, G, I, J, A, B, F, C

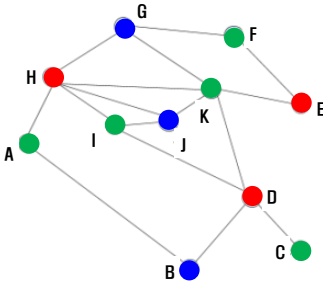
We can repeat the process with the second color Green

- K – color green
- G – don’ t color green, as it connects with K
- I – color green
- J – don’ t color green, as it connects with I
- A – color green
- B – don’ t color green, as it connects with A
- F – color green
- C – color green



4) Again, ignoring the coloured vertices, we are left with G, J, B Let’ s color it with Blue.

- G – color blue
- J – color blue
- B – color blue



Chromatic number of this graph is 3

98/99

# Welch-Powell 알고리즘

## Welch-Powell의 알고리즘

- ① 그래프  $G$ 의 정점의 차수가 내림차순(descending order)이 되게 배열한다(이 배열은 차수가 같은 정점이 여러 개 있을 수 있으므로 몇 가지 다른 순서가 존재할 수 있다).
- ② 배열의 첫 번째 정점은 첫 번째 색으로 착색하고 계속해서 배열의 순서대로 이미 착색된 정점과 인접하지 않은 정점을 모두 같은 색으로 착색한다.
- ③ 배열에서 먼저 나타나는 착색되지 않은 정점을 두 번째 색으로 착색하고 계속해서 배열의 순서대로 지금 착색하고 있는 색으로 이미 착색된 정점과 인접하지 않은 정점을 모두 착색한다.
- ④ 계속해서 위의 과정을 그래프의 모든 정점이 착색될 때까지 반복한다.

99/99

99

# 웰치-포웰 알고리즘

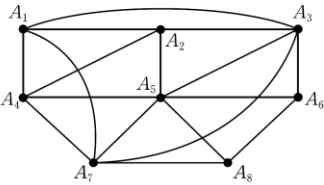
## 예제 7-21 웰치-포웰 알고리즘

오른쪽 그래프를 웰치-포웰 알고리즘으로 착색하라.

### 풀이

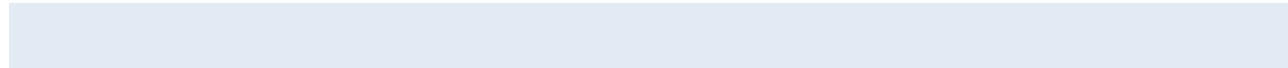
정점들을 내림차순으로 배열하면 다음과 같다.

- $A_5, A_3, A_7, A_1, A_2, A_4, A_6, A_8$
- $A_5$ 와  $A_1$ 이 먼저 첫 번째 색으로 착색되고, 다음  $A_3, A_4, A_8$ 이 두 번째 색으로 착색되며, 마지막으로  $A_7, A_2, A_6$ 이 세 번째 색으로 착색된다. 따라서 그래프는 3색이 가능하다.

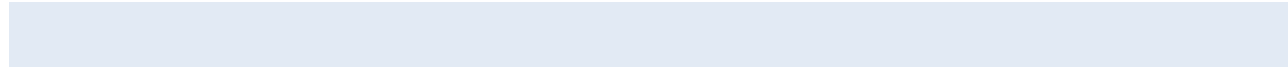


100/99

100



부 록



최단 경로 (Dijkstra's algorithm)

<교재 내용>

# 최단 경로 (Dijkstra's algorithm)

## 최단 경로

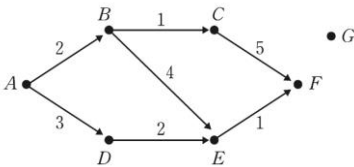
- 가중치 그래프를 이용한 두 정점 사이에 최단 경로(shortest path)를 찾는 문제를 생각해보자.
- 두 정점 사이에 최단 경로의 길이는 가중치들의 합 중에서 가장 작은 값을 가지는 경로를 말한다.
- 정점  $u$ 에서 정점  $v$  사이에 최단 경로를 두 정점 사이의 거리라고 한다.
- 만일 어떤 두 정점 사이에 최단 경로가 없다면 이때의 거리는 무한대( $\infty$ )라고 가정한다.
- 그러면 최단 경로를 어떻게 하면 찾을 수 있을까?  
그것에 대한 명백한 해답은 모든 경로에 대해서 거리를 계산하는 것이다.
- 그러나 이 방법은  $n$  개의 정점들이 있을 때 정점  $a$ 로부터 정점  $b$ 로의 경로는  $(n-2)!$  이상이 존재하므로 만일  $n = 30$ 일 경우에 경로를 컴퓨터로 계산한다고 하더라도 엄청난 시간을 요구할 것이다. 그러므로 모든 경우의 경로를 구해서 계산한다는 것은 무리이다.
- 이를 해결하는 알고리즘의 하나가 Dijkstra의 알고리즘이다
- Dijkstra는 네덜란드 사람으로 이론 물리학자이며 1972년 ACM으로부터 그 유명한 튜링(Alan Turing) 상을 수상했다. 튜링 상은 컴퓨터 분야에서의 노벨상이라고 불린다.

103/99

103

# 최단 경로 (Dijkstra's algorithm)

- [그림 7-10]에서  $A$ 부터  $F$ 까지의 최단 경로의 가중치는 6이며 정점  $A, D, E, F$ 로 구성됨을 알 수 있다.



[그림 7-10] 가중치 그래프

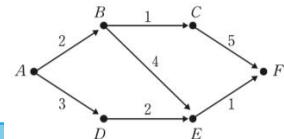
- [그림 7-10]의 가중치 그래프에서 정점  $A$ 로부터 다른 정점까지 최단 경로를 찾는 데 필요한 단계들을 먼저 살펴본 다음에 전체 알고리즘에 대해 알아보자.

104/99

104

# 최단 경로 (Dijkstra's algorithm)

- 가중치 그래프에서 각 정점  $x$ 에 대해서  $d[x]$ 는 **출발 정점  $A$ 로부터  $x$ 까지의 거리**를 나타낸다고 하자.
- 첫 번째로 초기값을 각  $d[x]$ 에 할당하고 정점을 한 번 이상 탐색하게 되면 그때  $d[x]$ 의 값을 바꾼다.
- 한 정점을 이미 탐색했다면 그 정점은 앞으로 찾아가는 정점에서 제외된다.
- [표 7-1]는 각각의 단계를 나타낸다.



[표 7-1] A로부터 최단 경로 찾기

| 단계 | 찾아갈 정점 | 정점에 대한 거리 |   |          |   |          |          |          | 탐색하지 않은 정점       |
|----|--------|-----------|---|----------|---|----------|----------|----------|------------------|
|    |        | A         | B | C        | D | E        | F        | G        |                  |
| 0  | A      | 0         | 2 | $\infty$ | 3 | $\infty$ | $\infty$ | $\infty$ | B, C, D, E, F, G |
| 1  | B      | 0         | 2 | 3        | 3 | 6        | $\infty$ | $\infty$ | C, D, E, F, G    |
| 2  | D      | 0         | 2 | 3        | 3 | 5        | $\infty$ | $\infty$ | C, E, F, G       |
| 3  | C      | 0         | 2 | 3        | 3 | 5        | 8        | $\infty$ | E, F, G          |
| 4  | E      | 0         | 2 | 3        | 3 | 5        | 6        | $\infty$ | F, G             |
| 5  | F      | 0         | 2 | 3        | 3 | 5        | 6        | $\infty$ | G                |

105/99

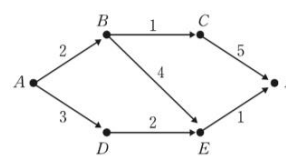
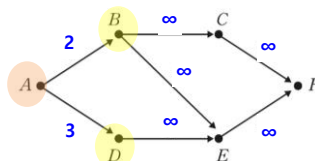
105

# 최단 경로 (Dijkstra's algorithm)

## ❖ 단계 0

- $X = A$ 일 때  $d[X]$ 의 값은 0,  $(A, X)$ 에 간선이 없으면  $\infty$ ,  $(A, X)$ 에 간선이 있으면 그 간선에 있는 가중치를 부여한다.
- 정점  $A$ 가 가장 가까운 정점이므로 이 정점을 제일 먼저 탐색하고 다른 정점들은 탐색하지 않는다.

| 단계 | 찾아갈 정점 | 정점에 대한 거리 |   |          |   |          |          |          | 탐색하지 않은 정점       |
|----|--------|-----------|---|----------|---|----------|----------|----------|------------------|
|    |        | A         | B | C        | D | E        | F        | G        |                  |
| 0  | A      | 0         | 2 | $\infty$ | 3 | $\infty$ | $\infty$ | $\infty$ | B, C, D, E, F, G |



106/99

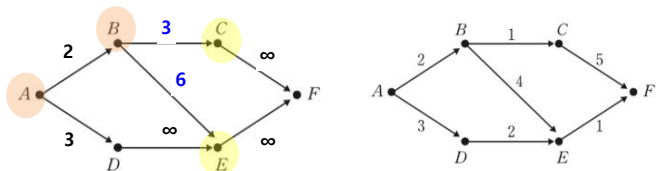
106

# 최단 경로 (Dijkstra's algorithm)

## ❖ 단계 1

- B가 A에 가장 가까운 탐색하지 않은 정점이므로 다음 정점으로 탐색하고, 탐색하지 않은 정점에서 제거한다.
- 탐색하지 않은 모든 Y에 대해서 B를 통과해서 Y까지의 새로운 거리가 이전에 계산된 거리보다 짧은가를 검사한다.
- C와 E에 도달할 수 있으므로 거리를 다시 계산해서 3과 6으로 쓴다.

| 단계 | 찾아갈 정점 | 정점에 대한 거리 |   |   |   |   |   |   | 탐색하지 않은 정점       |
|----|--------|-----------|---|---|---|---|---|---|------------------|
|    |        | A         | B | C | D | E | F | G |                  |
| 0  | A      | 0         | 2 | ∞ | 3 | ∞ | ∞ | ∞ | B, C, D, E, F, G |
| 1  | B      | 0         | 2 | 3 | 3 | 6 | ∞ | ∞ | C, D, E, F, G    |



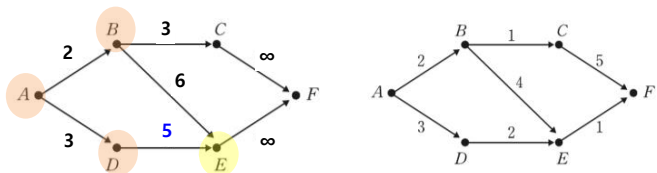
107/99

107

# 최단 경로 (Dijkstra's algorithm)

## ❖ 단계 2

- 나머지 탐색하지 않은 정점들 중에서 C와 D가 가장 가깝다.
- 임의로 D를 선택해서 탐색한다. E는 D를 통한 경로가 되므로 거리에 대한 값이 6에서 5로 바뀐다.



108/99

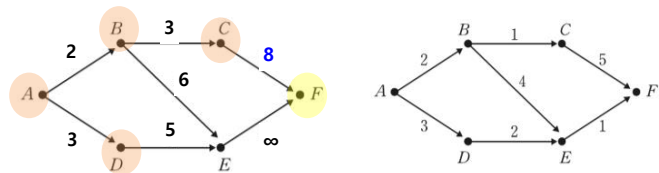
108

# 최단 경로 (Dijkstra's algorithm)

## ❖ 단계 3

- $C$ 는  $A$ 에 가장 가까운 정점이므로  $C$ 를 탐색한다.
- $F$ 가  $C$ 를 통해서 도달되므로 거리를 8로 바꾼다.

| 단계 | 찾아갈 정점 | 정점에 대한 거리 |   |          |   |          |          |          | 탐색하지 않은 정점       |
|----|--------|-----------|---|----------|---|----------|----------|----------|------------------|
|    |        | A         | B | C        | D | E        | F        | G        |                  |
| 0  | A      | 0         | 2 | $\infty$ | 3 | $\infty$ | $\infty$ | $\infty$ | B, C, D, E, F, G |
| 1  | B      | 0         | 2 | 3        | 3 | 6        | $\infty$ | $\infty$ | C, D, E, F, G    |
| 2  | D      | 0         | 2 | 3        | 3 | 5        | $\infty$ | $\infty$ | C, E, F, G       |
| 3  | C      | 0         | 2 | 3        | 3 | 5        | 8        | $\infty$ | E, F, G          |



109/99

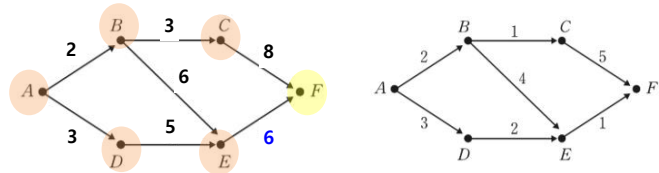
109

# 최단 경로

## ❖ 단계 4

- 다음에  $E$ 를 탐색하고  $F$ 로의 거리를 6으로 바꾼다.

| 단계 | 찾아갈 정점 | 정점에 대한 거리 |   |          |   |          |          |          | 탐색하지 않은 정점       |
|----|--------|-----------|---|----------|---|----------|----------|----------|------------------|
|    |        | A         | B | C        | D | E        | F        | G        |                  |
| 0  | A      | 0         | 2 | $\infty$ | 3 | $\infty$ | $\infty$ | $\infty$ | B, C, D, E, F, G |
| 1  | B      | 0         | 2 | 3        | 3 | 6        | $\infty$ | $\infty$ | C, D, E, F, G    |
| 2  | D      | 0         | 2 | 3        | 3 | 5        | $\infty$ | $\infty$ | C, E, F, G       |
| 3  | C      | 0         | 2 | 3        | 3 | 5        | 8        | $\infty$ | E, F, G          |
| 4  | E      | 0         | 2 | 3        | 3 | 5        | 6        | $\infty$ | F, G             |



110/99

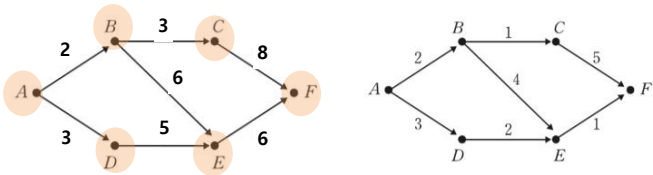
110

# 최단 경로 (Dijkstra's algorithm)

## ❖ 단계 5

- *F*를 탐색한다. 도달될 수 있는 정점이 없으므로 *G*는 탐색되지 않는다.

| 단계 | 찾아갈 정점 | 정점에 대한 거리 |   |   |   |   |   |   | 탐색하지 않은 정점       |
|----|--------|-----------|---|---|---|---|---|---|------------------|
|    |        | A         | B | C | D | E | F | G |                  |
| 0  | A      | 0         | 2 | ∞ | 3 | ∞ | ∞ | ∞ | B, C, D, E, F, G |
| 1  | B      | 0         | 2 | 3 | 3 | 6 | ∞ | ∞ | C, D, E, F, G    |
| 2  | D      | 0         | 2 | 3 | 3 | 5 | ∞ | ∞ | C, E, F, G       |
| 3  | C      | 0         | 2 | 3 | 3 | 5 | 8 | ∞ | E, F, G          |
| 4  | E      | 0         | 2 | 3 | 3 | 5 | 6 | ∞ | F, G             |
| 5  | F      | 0         | 2 | 3 | 3 | 5 | 6 | ∞ | G                |



111/99

111

## DFS/BFS

[출처] geeksforgeeks  
<https://www.geeksforgeeks.org/depth-first-search-or-dfs-for-a-graph/?ref=lbp>  
<https://www.geeksforgeeks.org/breadth-first-search-or-bfs-for-a-graph/?ref=lbp>

112/99

112

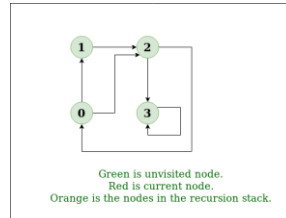


## Depth First Search or DFS for a Graph

- ❖ Depth First Traversal (or Search) for a graph is similar to Depth First Traversal of a tree. The only catch here is, unlike trees, graphs may contain cycles, a node may be visited twice. To avoid processing a node more than once, use a boolean visited array.

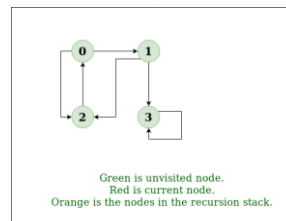
### ❖ Example

- Input:  $n = 4, e = 6$
- $0 \rightarrow 1, 0 \rightarrow 2, 1 \rightarrow 2, 2 \rightarrow 0, 2 \rightarrow 3, 3 \rightarrow 3$
- Output: DFS from vertex 1 : 1 2 0 3
- Explanation:
- DFS Diagram:



gif 파일

- Input:  $n = 4, e = 6$
- $2 \rightarrow 0, 0 \rightarrow 2, 1 \rightarrow 2, 0 \rightarrow 1, 3 \rightarrow 3, 1 \rightarrow 3$
- Output: DFS from vertex 2 : 2 0 1 3
- Explanation:
- DFS Diagram:



113

113/99

## Depth First Search or DFS for a Graph

- Following are implementations of simple Depth First Traversal. The C++ implementation uses adjacency list representation of graphs. STL 's list container is used to store lists of adjacent nodes.
- Solution:
  - ✓ Approach: Depth-first search is an algorithm for traversing or searching tree or graph data structures. The algorithm starts at the root node (selecting some arbitrary node as the root node in the case of a graph) and explores as far as possible along each branch before backtracking. So the basic idea is to start from the root or any arbitrary node and mark the node and move to the adjacent unmarked node and continue this loop until there is no unmarked adjacent node. Then backtrack and check for other unmarked nodes and traverse them. Finally print the nodes in the path.
- Algorithm:
  - ✓ Create a recursive function that takes the index of node and a visited array.
  - ✓ Mark the current node as visited and print the node.
  - ✓ Traverse all the adjacent and unmarked nodes and call the recursive function with index of adjacent node.

C++ 코드 : C07-04-DFS.cpp  
Python 코드 : C07-04-DFS.py

114

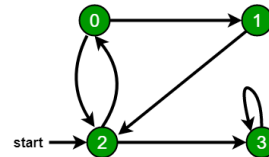
114/99

## Breadth First Search or BFS for a Graph

- ❖ Breadth First Traversal (or Search) for a graph is similar to Breadth First Traversal of a tree (See method 2 of this post). The only catch here is, unlike trees, graphs may contain cycles, so we may come to the same node again. To avoid processing a node more than once, we use a boolean visited array. For simplicity, it is assumed that all vertices are reachable from the starting vertex.

- ❖ Example

- In the following graph, we start traversal from vertex 2. When we come to vertex 0, we look for all adjacent vertices of it. 2 is also an adjacent vertex of 0. If we don't mark visited vertices, then 2 will be processed again and it will become a non-terminating process. A Breadth First Traversal of the following graph is 2, 0, 3, 1.



- The implementation uses adjacency list representation of graphs. STL's list container is used to store lists of adjacent nodes and queue of nodes needed for BFS traversal.

C++ 코드 : C07-05-BFS.cpp  
Python 코드 : C07-05-BFS.py

115/99

115



## Welch-Powell Algorithms

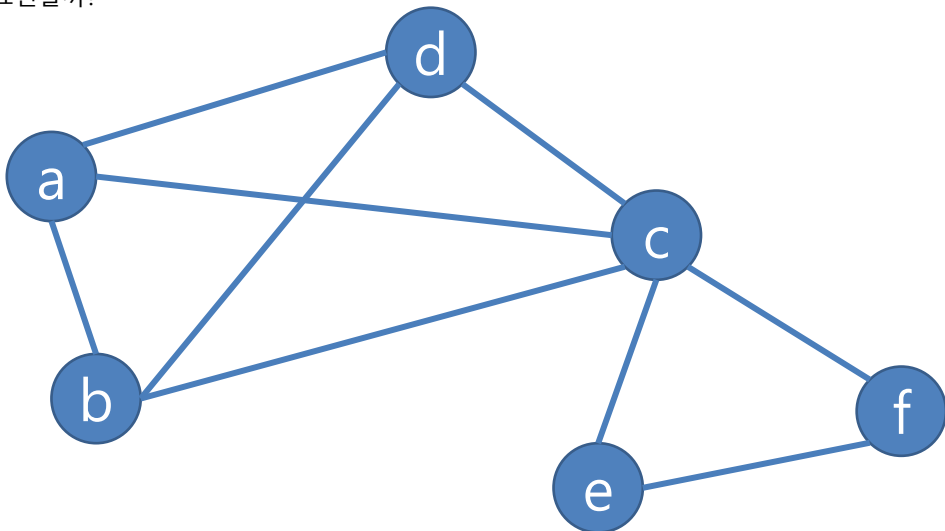
이산수학  
천세진

116

- ① 그래프  $G$ 의 정점의 차수가 내림차순(descending order)이 되게 배열한다(이 배열은 차수가 같은 정점이 여러 개 있을 수 있으므로 몇 가지 다른 순서가 존재할 수 있다).
- ② 배열의 첫 번째 정점은 첫 번째 색으로 착색하고 계속해서 배열의 순서대로 이미 착색된 정점과 인접하지 않은 정점을 모두 같은 색으로 착색한다.
- ③ 배열에서 먼저 나타나는 착색되지 않은 정점을 두 번째 색으로 착색하고 계속해서 배열의 순서대로 지금 착색하고 있는 색으로 이미 착색된 정점과 인접하지 않은 정점을 모두 착색한다.
- ④ 계속해서 위의 과정을 그래프의 모든 정점이 착색될 때까지 반복한다.

117

어떻게 코드로 표현할까?



118

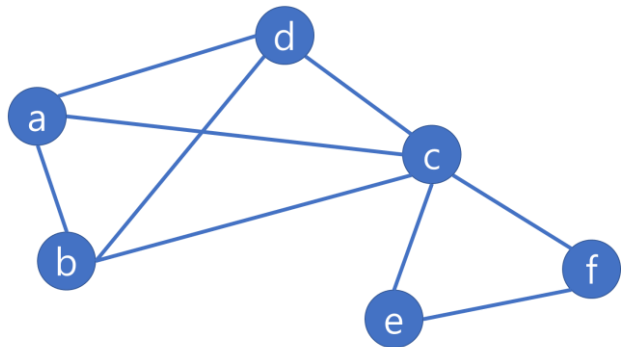
❑ graph = {}

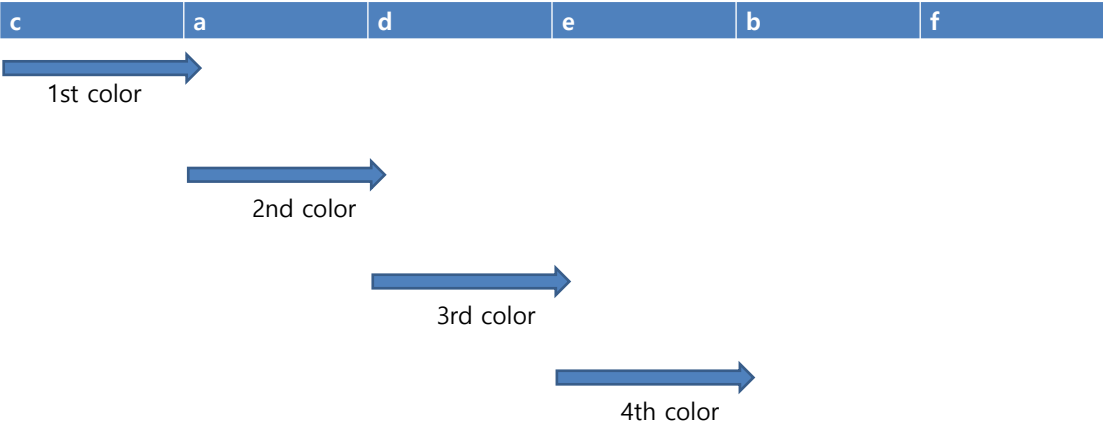
❑ 노드는 key로 표현, 이웃노드는 리스트로 표현

○ 'a' : [ 'b', 'c', 'd' ]

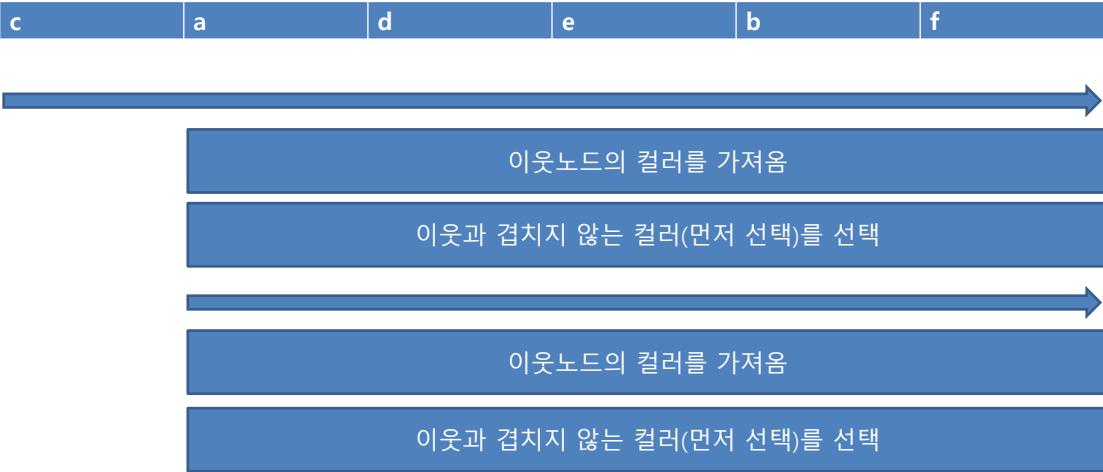
○ 'b' : [ 'c', 'f' ]

○ (Un-)Directed 그래프 모두 표현가능





121



122

❑ `sorted(data, key=lambda x: func, reverse=True/False)`

123

❑ `color_map = {} // 'c' : 0, 'a' : 1`

❑ `for each neighbor_node,`  
    `if neighbor_node in color_map:`  
        `color = color_map[neighbor_node]`  
        `available_colors[color] = False # 이웃인 노드들은 False`

124

```

❑ for each neighbor_node,
    if neighbor_node in color_map:
        color = color_map[neighbor_node]
        available_colors[color] = False # 이웃인 노드들은 False

    for color, is_colorable in enumerate(available_colors):
        if is_colorable: # 색칠 가능할 때
            color_map[node] = color
            break

```

125

```

❑ return color_map

```

126

```

def color_nodes(graph):
    nodes = sorted(list(graph.keys()), key=lambda x: len(graph[x]), reverse=True)
    color_map = {}

    for node in nodes:
        available_colors = [True] * len(nodes)
        for neighbor in graph[node]:
            if neighbor in color_map:
                color = color_map[neighbor]
                available_colors[color] = False
        for color, is_colorable in enumerate(available_colors):
            if is_colorable:
                color_map[node] = color
                break

    return color_map

```

127

```

def color_nodes(graph):
    nodes = sorted(list(graph.keys()), key=lambda x: len(graph[x]), reverse=True)
    color_map = {}

    for node in nodes:
        available_colors = [True] * len(nodes)
        for neighbor in graph[node]:
            if neighbor in color_map:
                color = color_map[neighbor]
                available_colors[color] = False
        for color, is_colorable in enumerate(available_colors):
            if is_colorable:
                color_map[node] = color
                break

    return color_map

```

**Generator expression using set**

128



```

def color_nodes(graph):
    nodes = sorted(list(graph.keys()), key=lambda x: len(graph[x]), reverse=True)
    color_map = {}

    for node in nodes:
        available_colors = [True] * len(nodes)
        for neighbor in graph[node]:
            if neighbor in color_map:
                color = color_map[neighbor]
                available_colors[color] = False
            for color, is_colorable in enumerate(available_colors):
                if is_colorable:
                    color_map[node] = color
                    break

    return color_map

```

**next ( generator expression with a condition)**